

Optimizing Attention for Modern Hardware

Supervisor: Prof. Dr. Khaled Elshafey

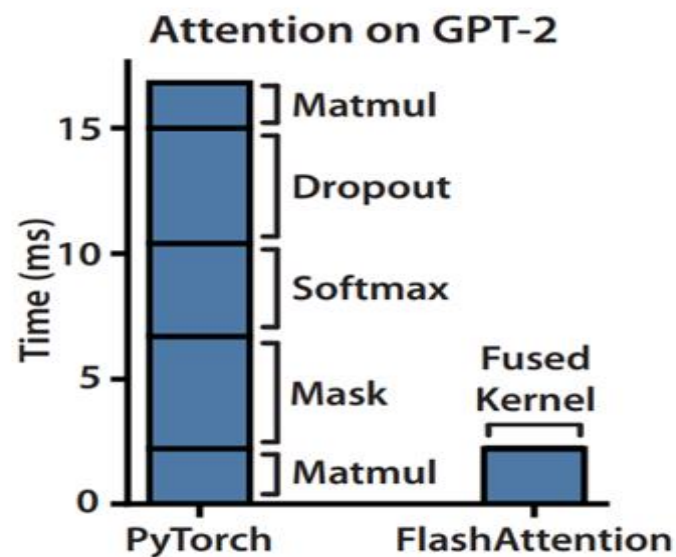
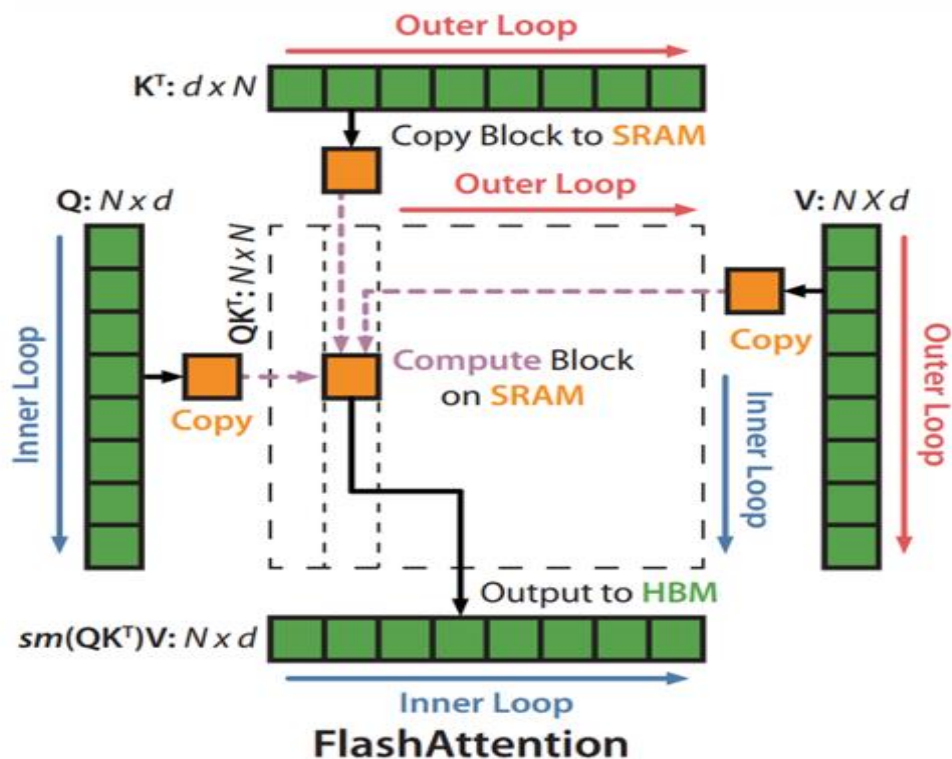
Instructor: Mohamed Gomaa



Goals:

Make CUDA and GPUs less magic

Understand how to make fast algorithms



Outline

- GPU Architecture
- GPU Programming
- GPU Bottlenecks
- GPU Acceleration
- LLM (Transformer)
- GPU Acceleration for Transformer

Outline

- GPU Architecture
- GPU Programming
- GPU Bottlenecks
- GPU Acceleration
- LLM (Transformer)
- GPU Acceleration for Transformer

GPU Architecture



Nvidia RTX A600
(GA102)
84 SMs
(Streaming
Multiprocessors)
12 memory
controller (32bit
ea.) total 384bit.
6MB L2 cache.

Streaming Multiprocessor

4 partitions per SM, each with 32 cores → 32 threads each

128 cores per SM

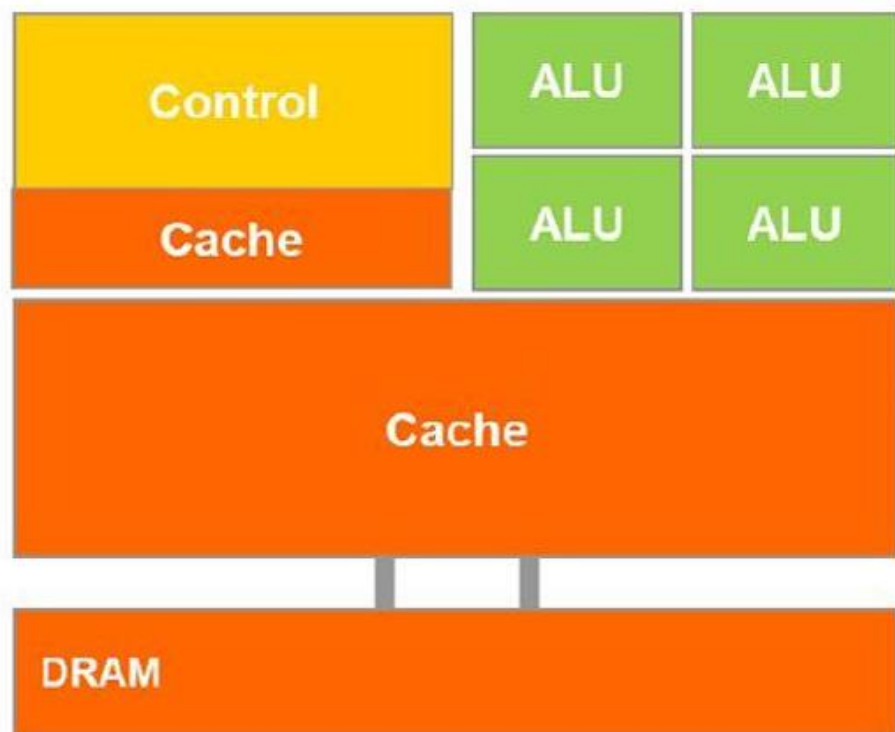
64KB register per partition (fastest to access)

128KB shared L1 cache per SM

128 FP32 operations in one cycle



CPU vs GPU



CPU

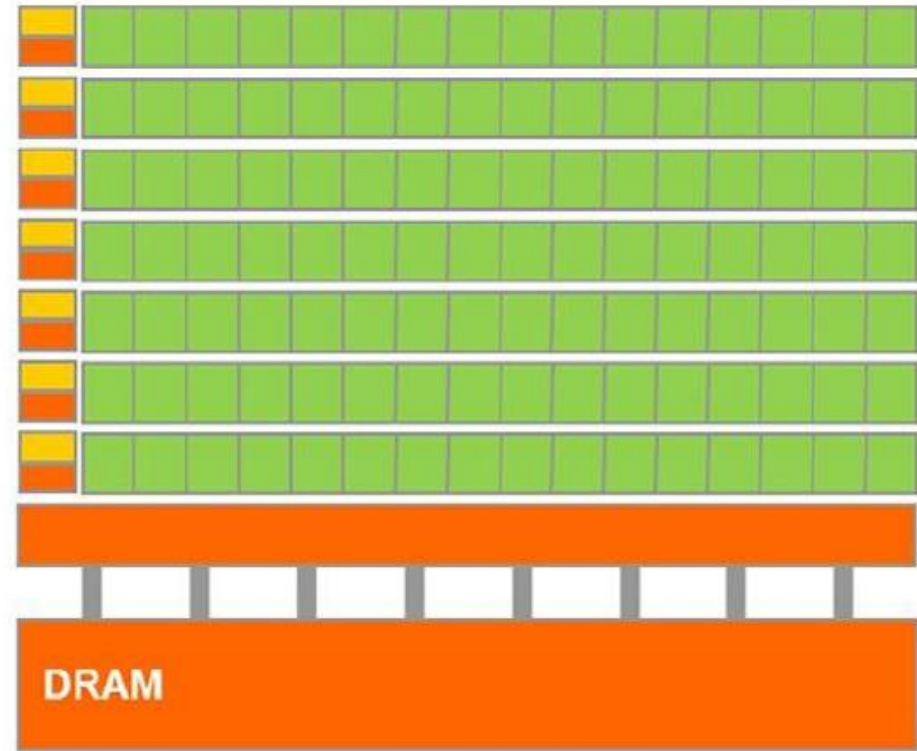
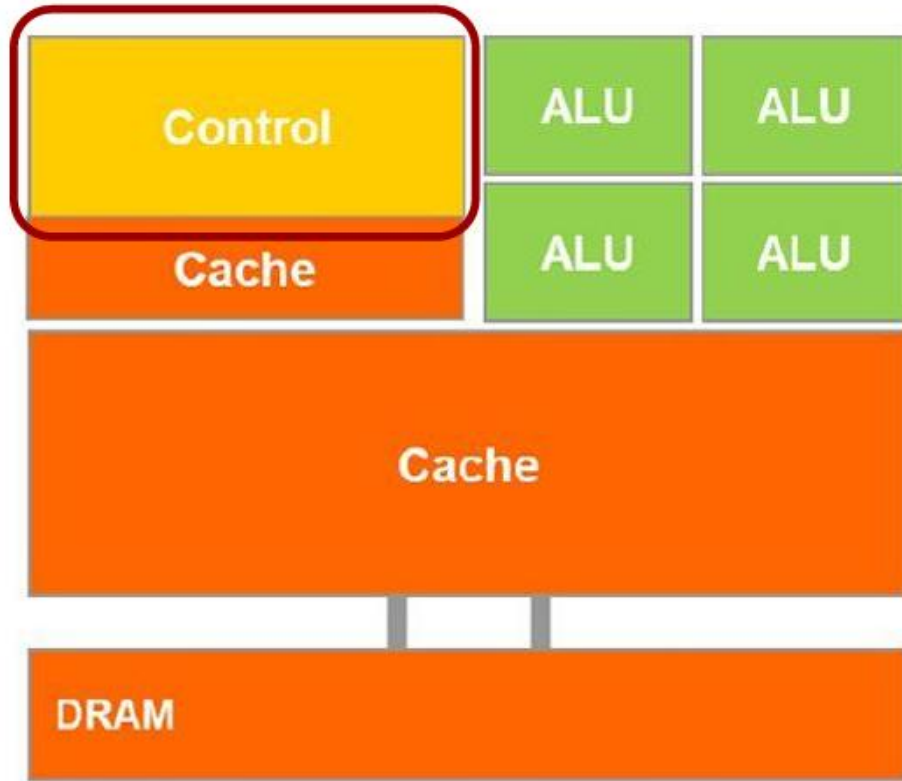
(latency-oriented design)



GPU

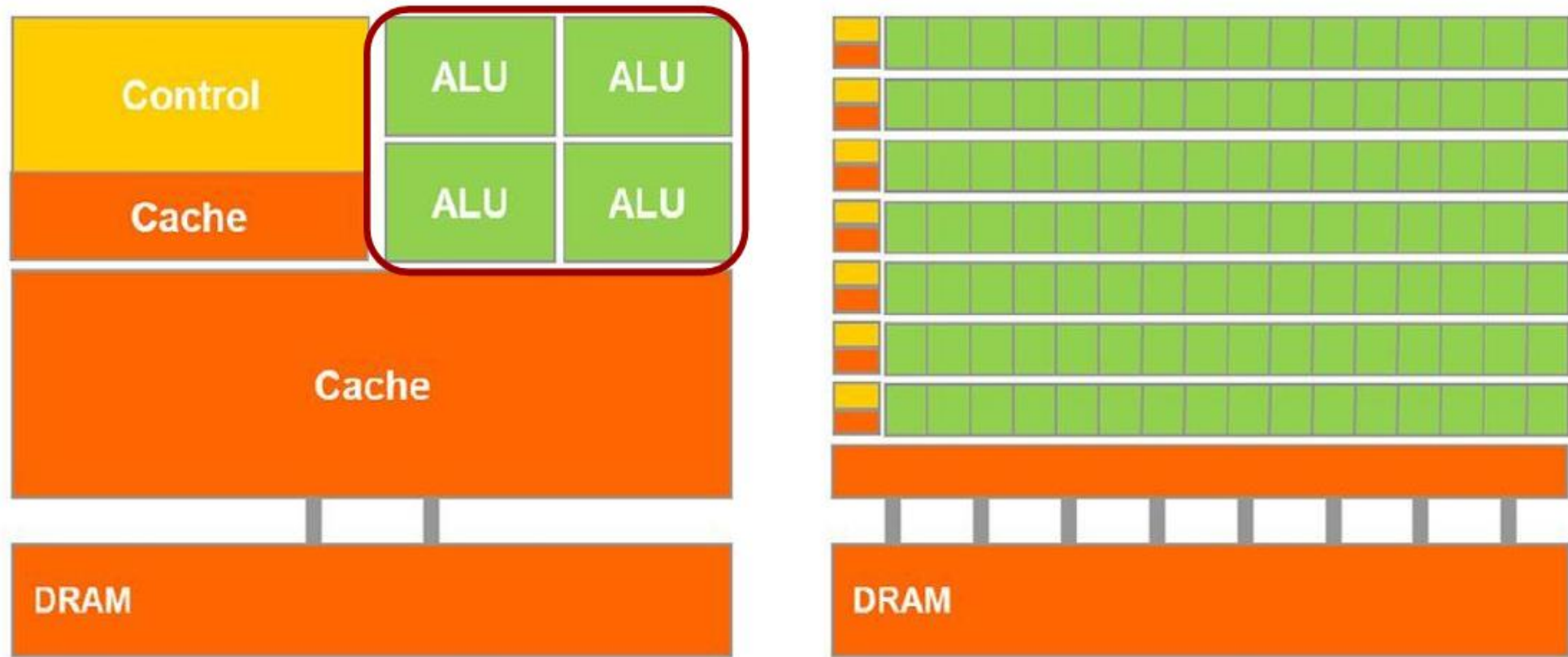
(throughput-oriented design)

CPU vs GPU



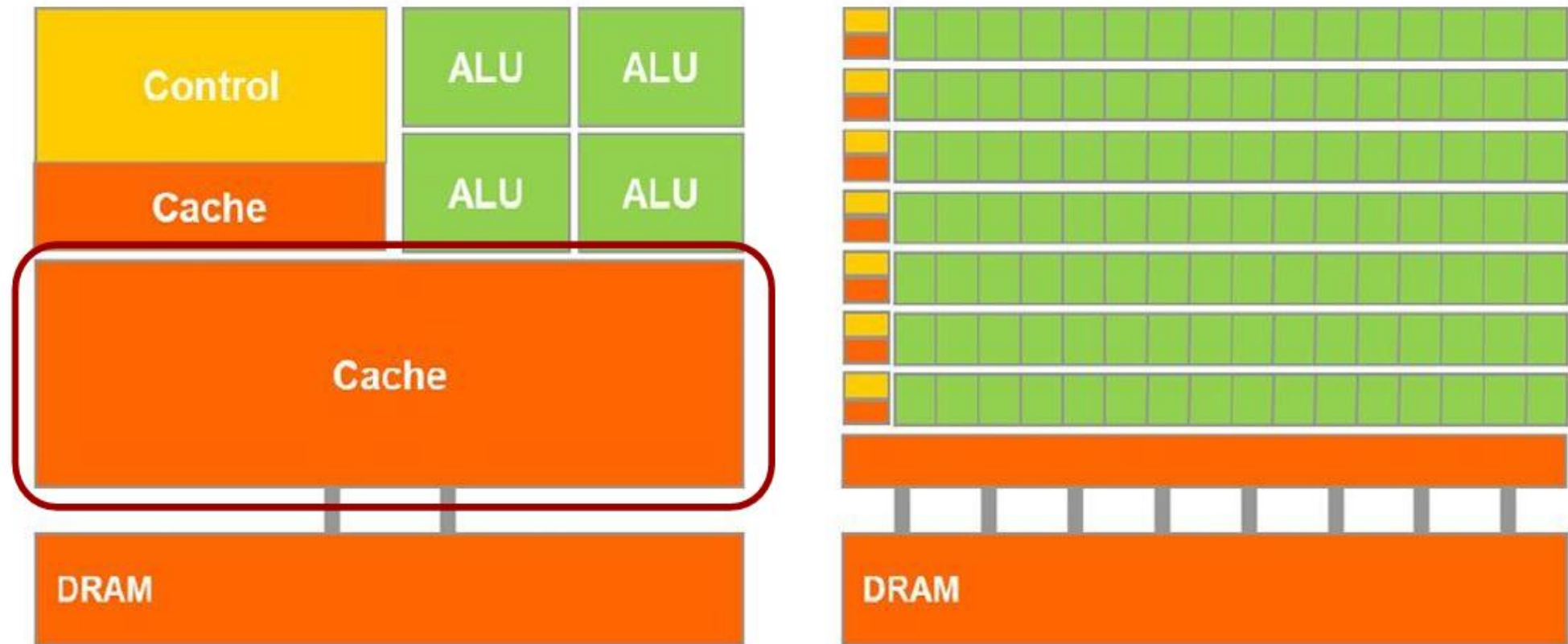
CPU Control unit: More complex control flow + sophisticated instruction processing

CPU vs GPU



CPU ALUs/FPU's: Less in number, but individually more capable

CPU vs GPU



CPU Caches: Deeper, more complex -> more chip area + power

CPU vs GPU

	NVIDIA H100	Intel Xeon Platinum 8480C
Peak FP64 TeraFLOPs/s	34	4.3
Memory B/W (GB/sec)	3350	307
DRAM latency (ns)	373	85

Outline

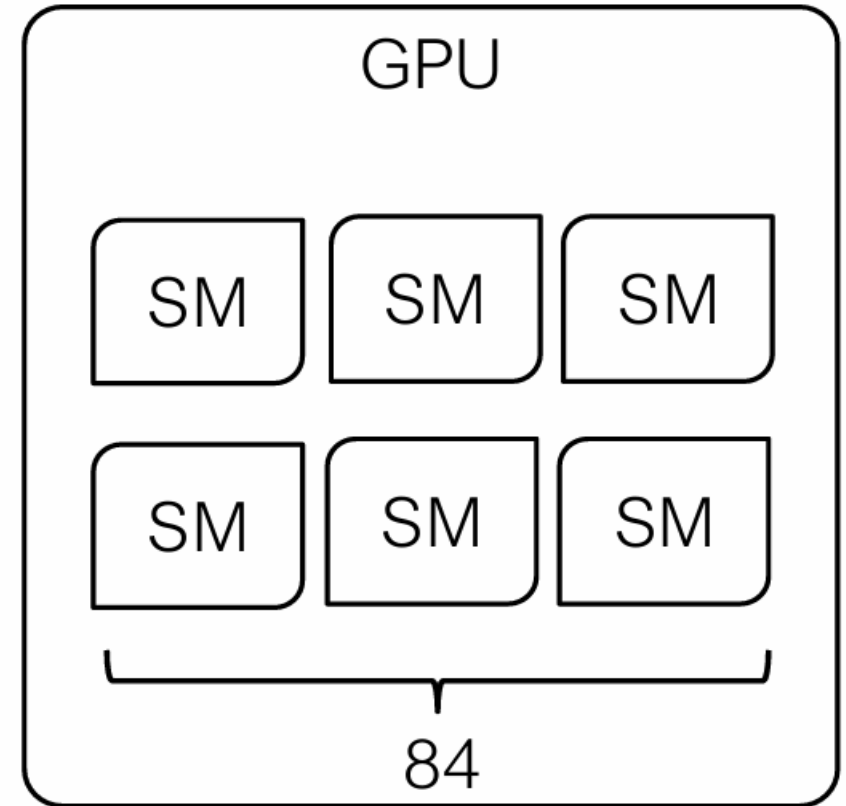
- GPU Architecture
- GPU Programming
- GPU Bottlenecks
- GPU Acceleration
- LLM (Transformer)
- GPU Acceleration for Transformer

GPU Programming Model

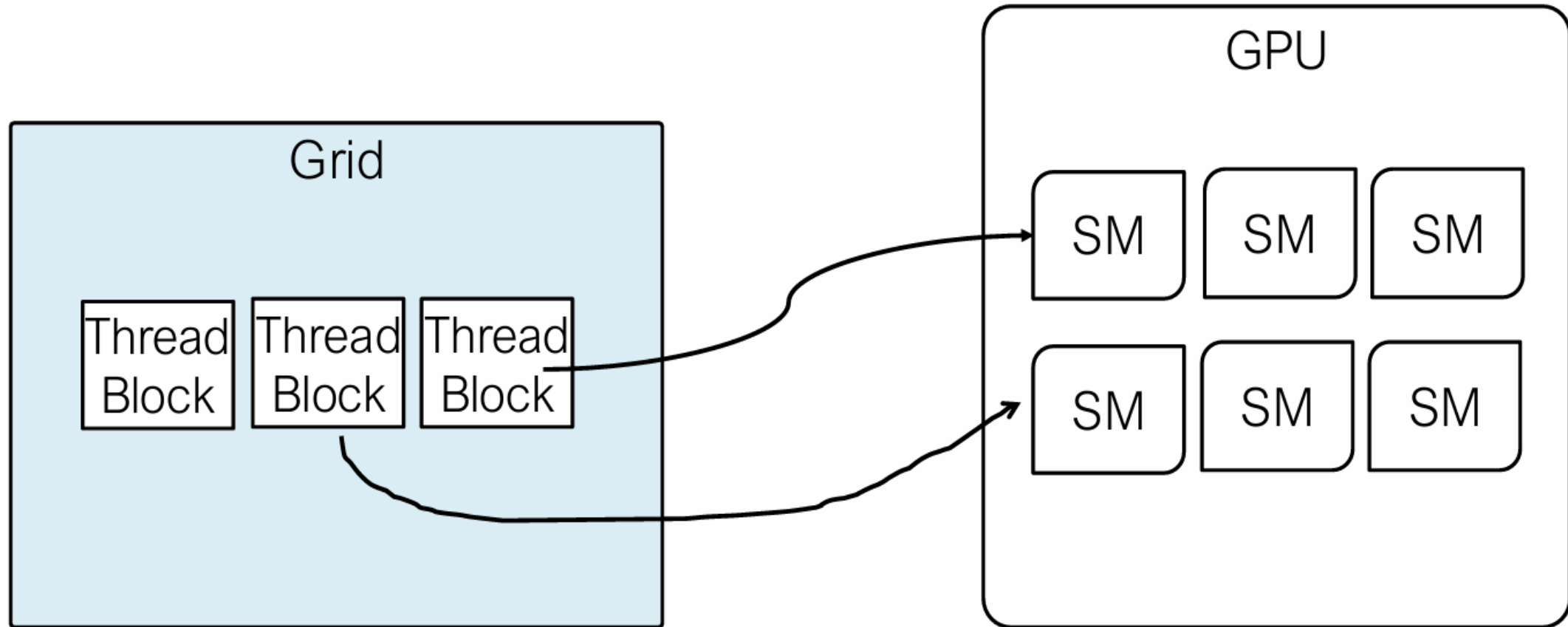
- CPU – host
 - Run normal program (C++)
- GPU – device
 - Run cuda kernel code
- CUDA: one part runs on CPU, one part runs on GPU
- Needs to move data between system memory and GPU memory

SIMT Execution on GPU

- Single Instruction Multiple Threads
- Threads are grouped into Thread Blocks
- Thread Blocks are grouped into Grid
- Kernel executed as Grid of Blocks of Threads



How instructions are executed on GPU

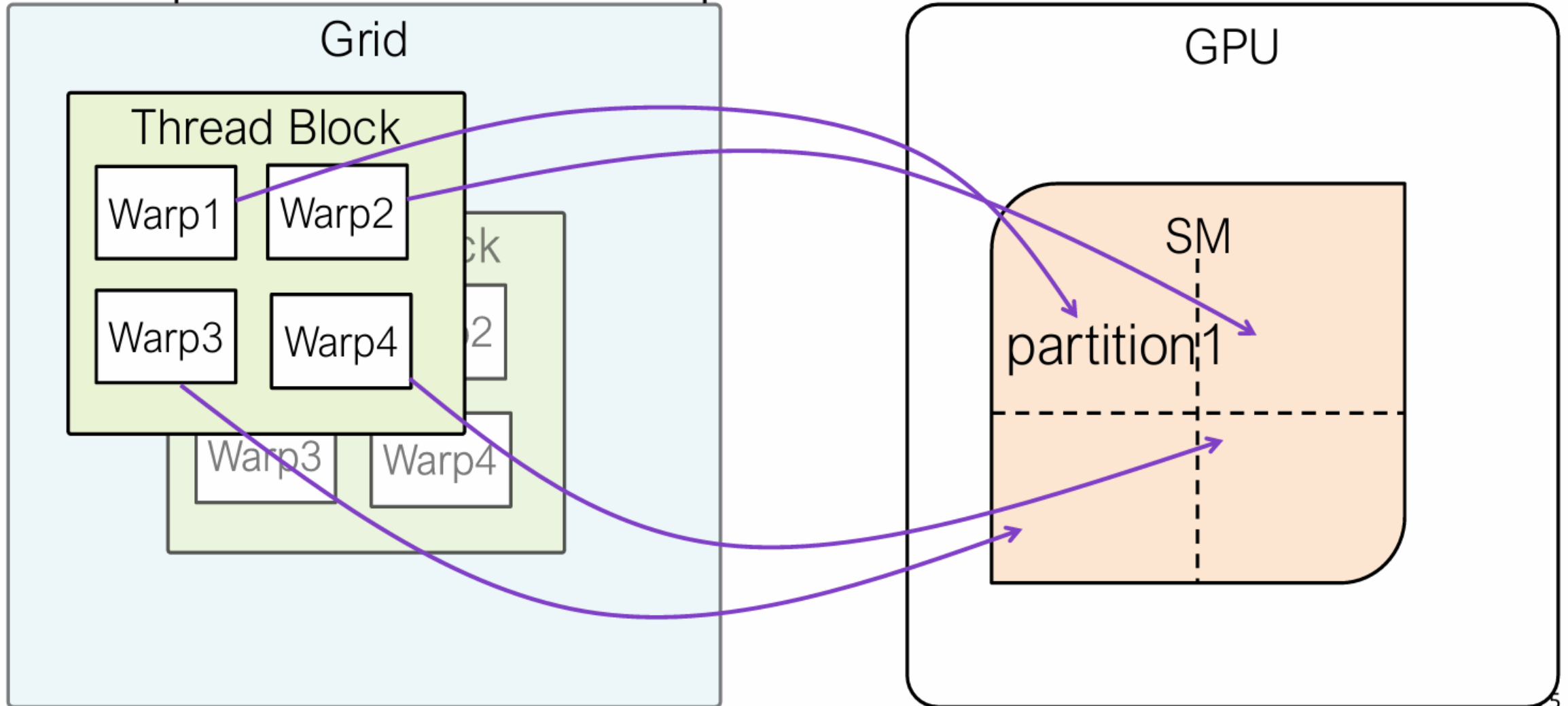


How Kernel Threads are Executed

- SM partition a thread block into warps
- Warp is the unit of GPU creating, managing, scheduling and executing threads
- Each warp contains **32** threads (why 32?)
 - Start at same program address
 - Have own program counter and registers
 - Execute one common instruction at a cycle
 - Can branch and execute independently

Executing one Thread block on one SM

4 warps can be executed in parallel at one time on each SM

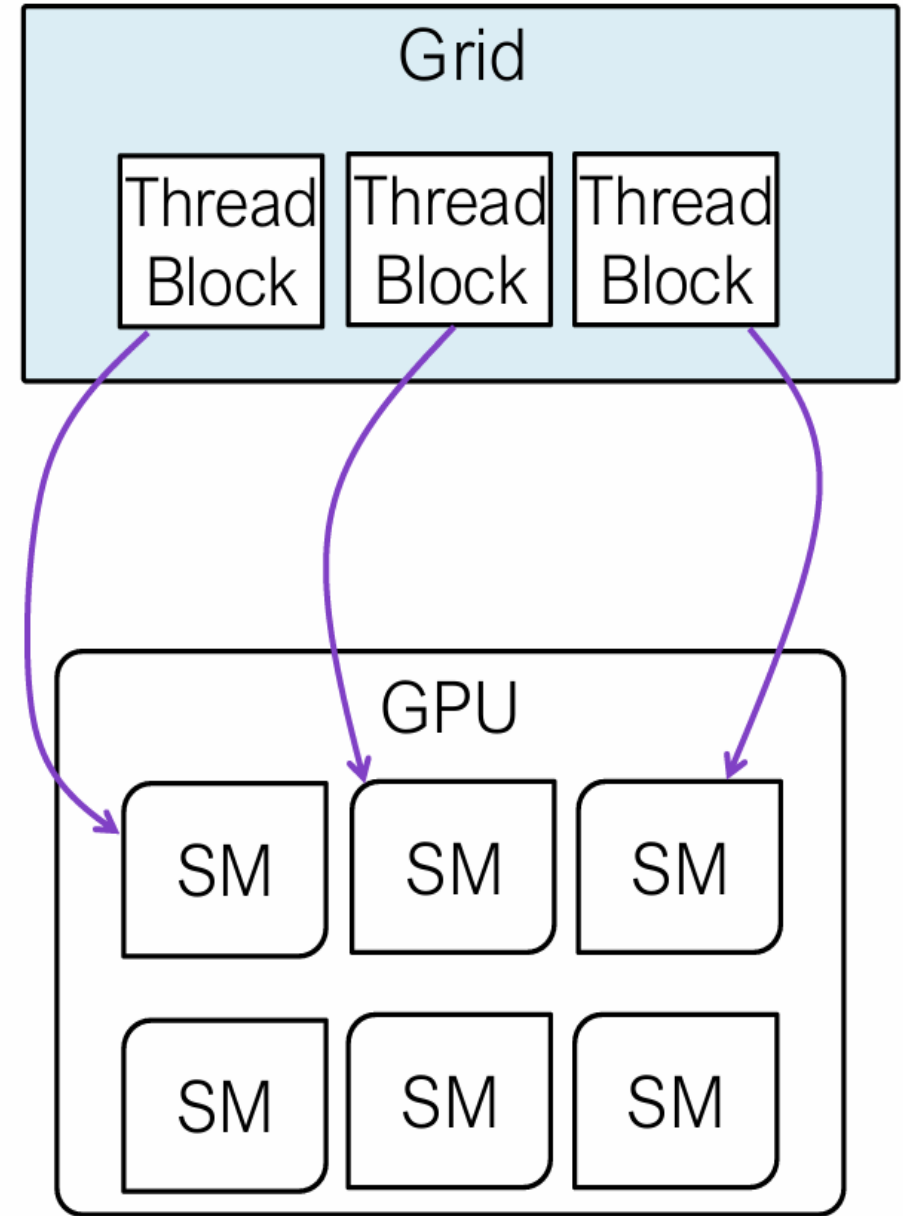


CUDA Kernel

- Each kernel is a function (program) that runs on GPU
- Program itself is serial
- Can simultaneously run many (10k) threads at the same time
- Using thread index to compute on right portion of data

Running GPU kernel

- CPU invokes kernel grid
- Thread blocks in grid distributed to SMs
- Execute concurrently
 - Each SM runs multiple thread blocks
 - Each core runs one thread from one thread block



CUDA Operations

- CPU allocates GPU memory: `cudaMalloc`
- CPU copies data to GPU memory (host to device): `cudaMemcpy`
- CPU launches GPU kernels
- CPU copies results from GPU (device to host): `cudaMemcpy`
- Freeing GPU memory `cudaFree`

Defining Functions to be executed on GPU

Define kernel function, `__global__`

```
__global__ void VecAddKernel(int* A, int* B, int* C, int n) {  
    int i = blockDim.x * blockIdx.x + threadIdx.x;  
    if (i < n) {  
        C[i] = A[i] + B[i];  
    }  
}  
  
int main() {  
    VecAddKernel<<<1, N>>>(A, B, C, N);  
}
```

Calling CUDA Kernel from CPU

Running kernels on GPU

```
// n: the size of the vector  
int n = 1024;  
int threads_per_block = 256;  
int num_blocks = (n + threads_per_block - 1) /  
    threads_per_block;  
VecAddKernel<<<num_blocks, threads_per_block>>>(dA, dB, dC, n);
```

CUDA Code Examples for Matrix Computation

```
__global__ void MatAddKernel(float* A, float* B, float* C, int N) {  
    int i = blockIdx.x * blockDim.x + threadIdx.x;  
    int j = blockIdx.y * blockDim.y + threadIdx.y;  
    C[i * N + j] = A[i * N + j] + B[i * N + j];  
}
```

```
int main() {  
    dim3 threads_per_block(2, 4, 8);  
    dim3 blocks_per_grid(2, 3, 4);  
    fullKernel<<<blocks_per_grid, threads_per_block>>>(some_input,  
    some_output);  
}
```

24 blocks per grid

64 threads per block

1536 threads in total

can you run this simultaneously on A6000?

Vector Addition

```
void VecAddCUDA(int* Acpu, int* Bcpu, int* Ccpu, int n) {  
    int *dA, *dB, *dC;  
    cudaMalloc(&dA, n * sizeof(int));  
    cudaMalloc(&dB, n * sizeof(int));  
    cudaMalloc(&dC, n * sizeof(int));  
    cudaMemcpy(dA, Acpu, n * sizeof(int), cudaMemcpyHostToDevice);  
    cudaMemcpy(dB, Bcpu, n * sizeof(int), cudaMemcpyHostToDevice);  
    int threads_per_block = 256;  
    int num_blocks = (n + threads_per_block - 1) / threads_per_block;  
    VecAddKernel<<<num_blocks, threads_per_block>>>(dA, dB, dC, n);  
    cudaMemcpy(Ccpu, dC, n * sizeof(int), cudaMemcpyDeviceToHost);  
    cudaFree(dA);  
    cudaFree(dB);  
    cudaFree(dC);  
}
```

Matrix Addition

```
void MatAddCUDA(int* Acpu, int* Bcpu, int* Ccpu, int n) {  
    int *dA, *dB, *dC;  
    cudaMalloc(&dA, n * n * sizeof(int));  
    cudaMalloc(&dB, n * n * sizeof(int));  
    cudaMalloc(&dC, n * n * sizeof(int));  
    cudaMemcpy(dA, Acpu, n * n * sizeof(int), cudaMemcpyHostToDevice);  
    cudaMemcpy(dB, Bcpu, n * n * sizeof(int), cudaMemcpyHostToDevice);  
    int THREADS = 32;  
    int BLOCKS = (n + THREADS - 1) / THREADS;  
    dim3 threads(THREADS, THREADS); // should be <= 1024  
    dim3 blocks(BLOCKS, BLOCKS);  
    MatAddKernel<<<blocks, threads>>>(dA, dB, dC, n);  
    cudaMemcpy(Ccpu, dC, n * n * sizeof(int), cudaMemcpyDeviceToHost);  
    cudaFree(dA);  
    cudaFree(dB);  
    cudaFree(dC);  
}
```

Outline

- GPU Architecture
- GPU Programming
- **GPU Bottlenecks**
- GPU Acceleration
- LLM (Transformer)
- GPU Acceleration for Transformer

Background: GPU Compute Model & Memory Hierarchy

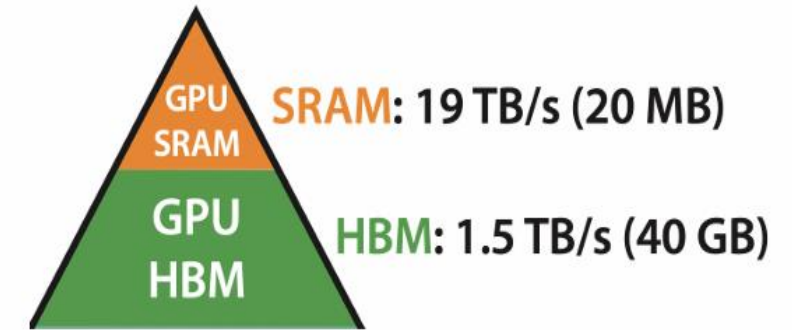
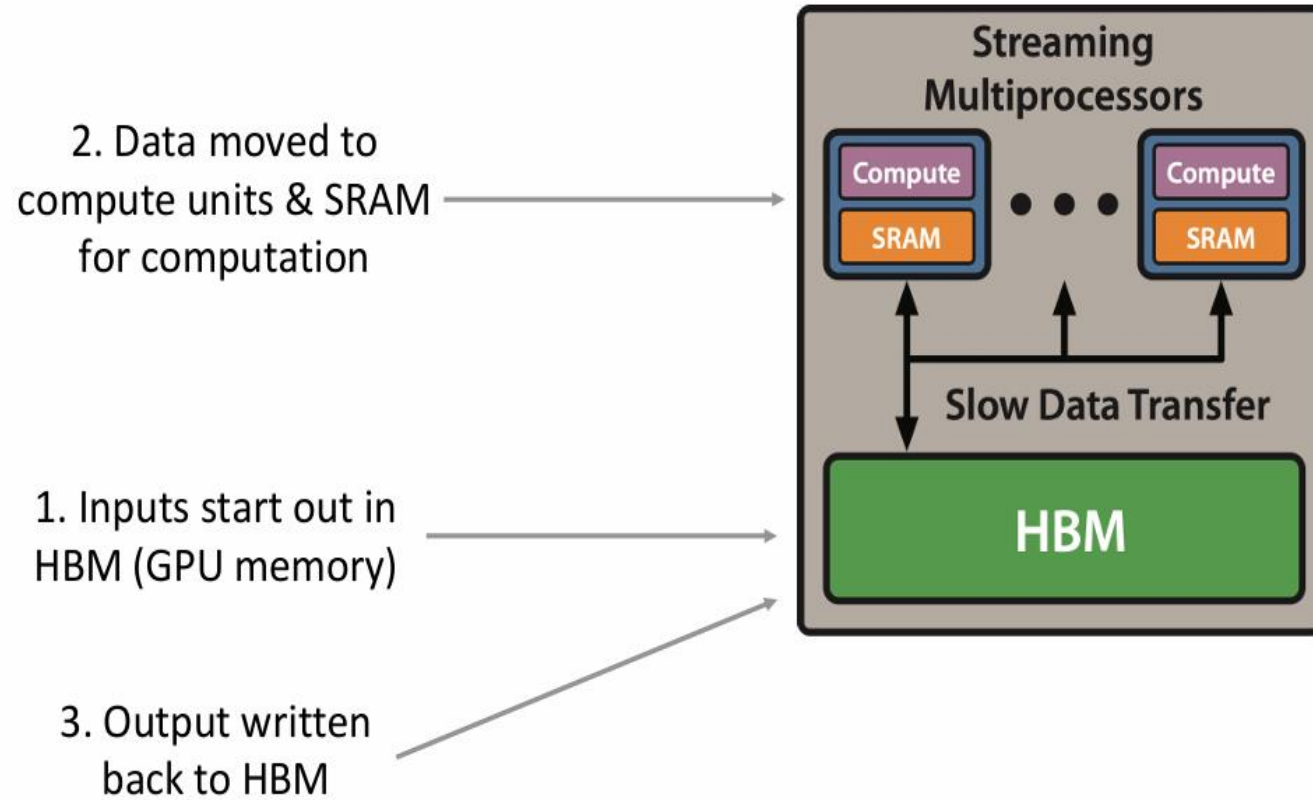
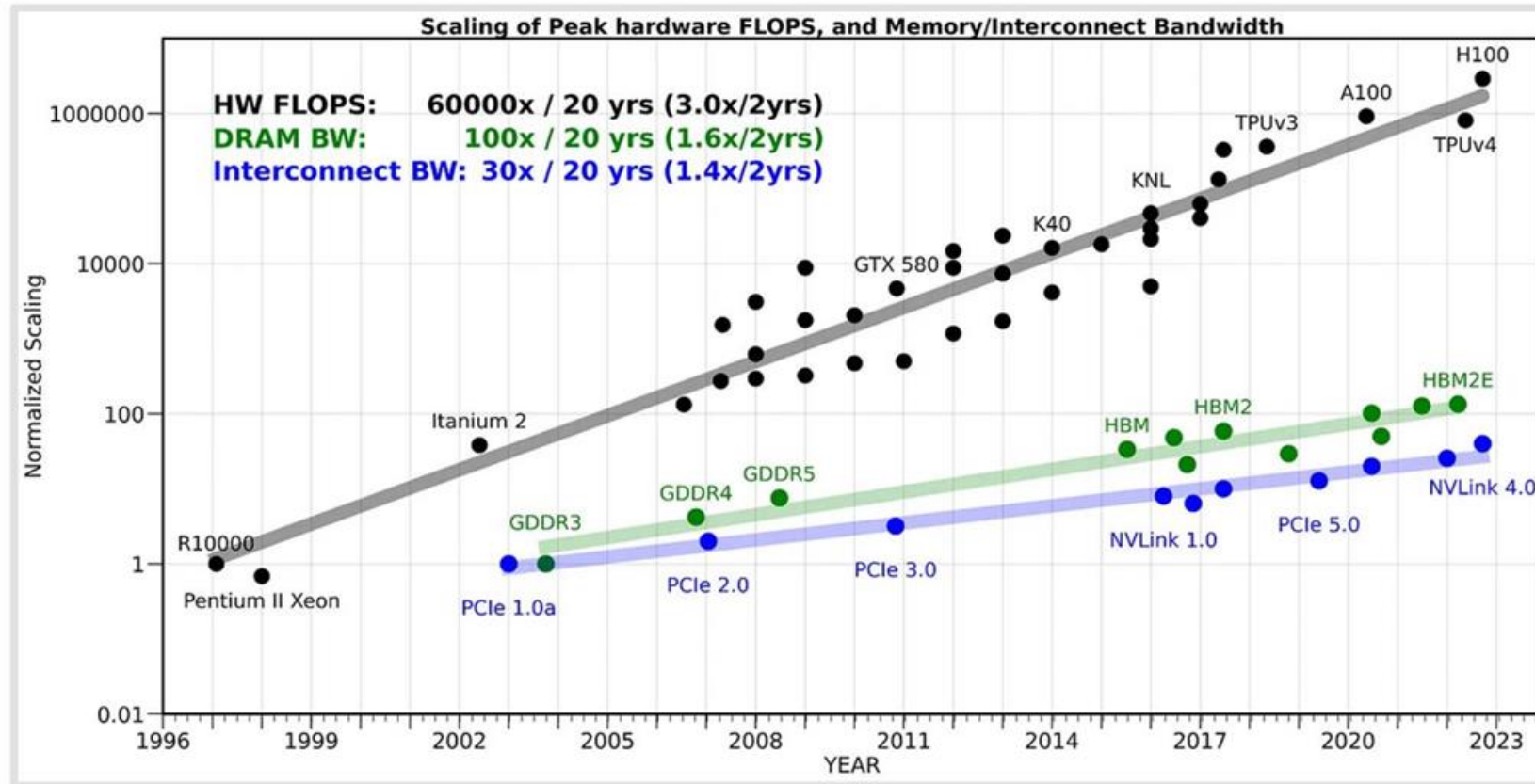


TABLE IV
THE MEMORY ACCESSES LATENCIES

Memory type	CPI (cycles)
Global memory	290
L2 cache	200
L1 cache	33
Shared Memory (ld/st)	(23/19)

Compute scaling is faster than memory scaling



<https://medium.com/riselab/ai-and-memory-wall-2cb4265cb0b8>

FLOPs scale faster than memory – it's hard to keep our compute units fed with data!

Memory Access Efficiency is Critical

A100 80G PCIe	
FP32	19.5 TFLOPS
Tensor Float 32	156 TFLOPS
GPU Memory Bandwidth	1,935 GB/s

- How many FP32 operations per second
 - may also be bounded by memory load/store
- Compute-to-global-memory-access-ratio
 - the number of FLOPs performed for each byte access from the GPU global memory

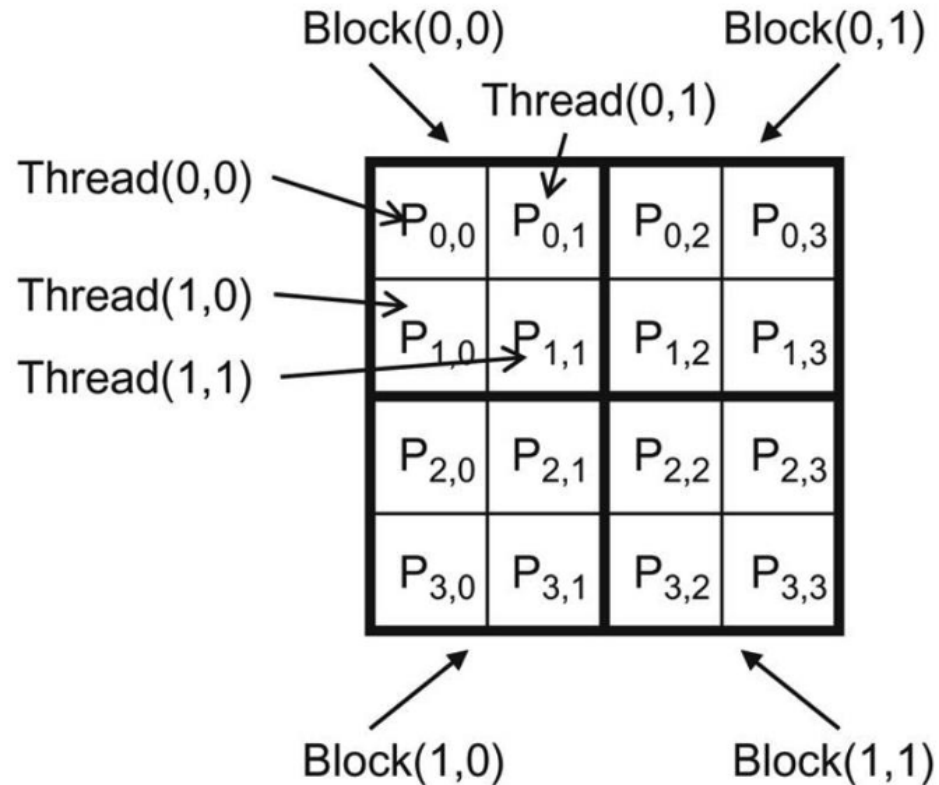
Memory Access in Matrix Multiplication

- Grid
 - four thread blocks (2x2)
 - each block with 4 threads (2x2)
- Every thread is responsible for calculating one element of result matrix P.

dim3 dimBlock(2, 2);

dim3 dimGrid(2, 2);

BLOCK_WIDTH = 2



Example: simple matrix multiplication

```
__global__ void MatMulKernel(float *a, float *b, float *c, int N) {  
    // Compute each thread's global row and col index -> output: (i, j)  
    int row = blockIdx.y * blockDim.y + threadIdx.y;  
    int col = blockIdx.x * blockDim.x + threadIdx.x;  
  
    if (row >= N || col >= N) return;  
    float Pvalue = 0.0;  
    for (int k = 0; k < N; k++) {  
        Pvalue += a[row * N + k] * b[k * N + col];  
    }  
    c[row * N + col] = Pvalue;  
}
```

1 FP32 multiply

1 FP32 add

2 global memory access
of FP32 (=4B)

compute-to-global-
memory-access:

$2 / (2 * 4) = 0.25 \text{ FLOP/B}$

Peak Computation on A100

A100 80G PCIe	
FP32	19.5 TFLOPS
Tensor Float 32	156 TFLOPS
GPU Memory Bandwidth	1,935 GB/s

- How much computation is based on memory access?
 - $1935\text{G} * 0.25 \text{ FLOP/B} = 483.75 \text{ GFLOPs}$
 - $483.75\text{GFLOPs} = 2.48\%$ of peak FP32 ops
 - $483.75\text{GFLOPs} = 0.3\%$ of peak Tensor FP32 ops
- Memory-bound program
 - computation limited by data transfer rate from memory

Loading vs Computing

```
C[i] = A[i] + B[i];
```

GPU Instructions:

```
ld.global.f32 %f1, [%rd1]; // Load A[i] 500 cycle
```

```
ld.global.f32 %f2, [%rd2]; // Load B[i] 500 cycle
```

```
add.f32 %f3, %f1, %f2; // Perform fp32 addition 1 cycle
```

```
st.global.f32 [%rd3], %f3; // Store result
```

Loading data takes more time than actual computation!

Outline

- GPU Architecture
- GPU Programming
- GPU Bottlenecks
- **GPU Acceleration**
- LLM (Transformer)
- GPU Acceleration for Transformer

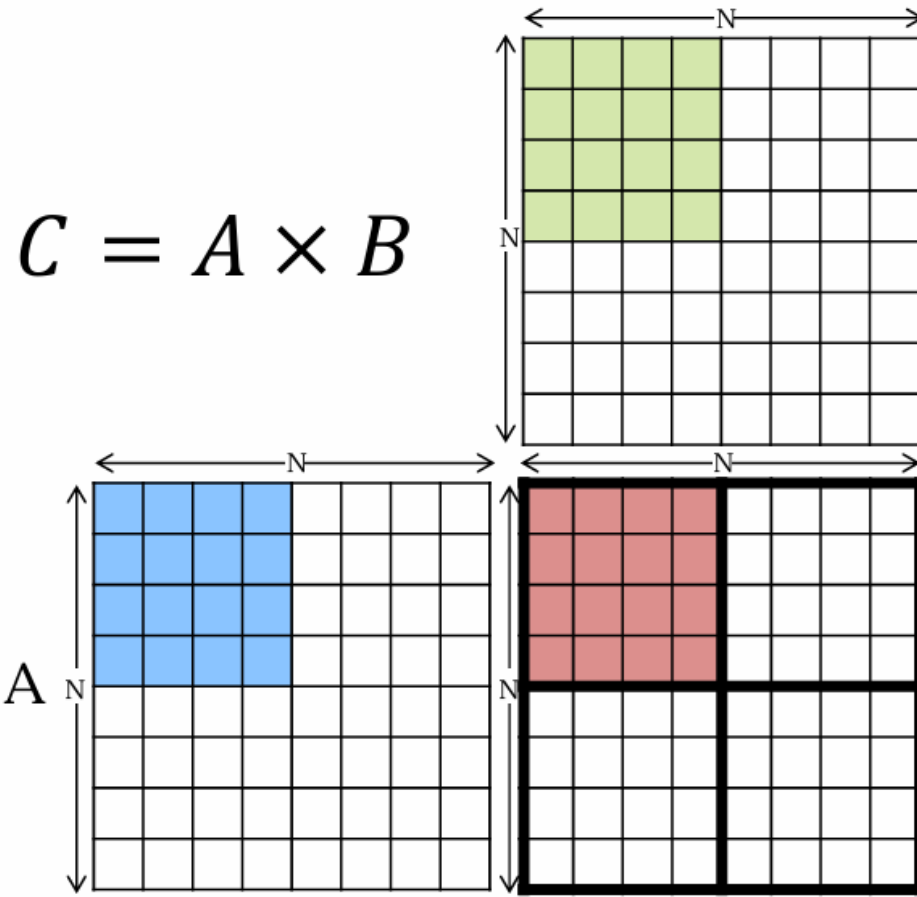
GPU Acceleration

- Tiling
- Kernel Fusion
- Low precision computation
- Recomputation

GPU Acceleration

- Tiling
- Kernel Fusion
- Low precision computation
- Recomputation

Tiling for Matrix Multiplication



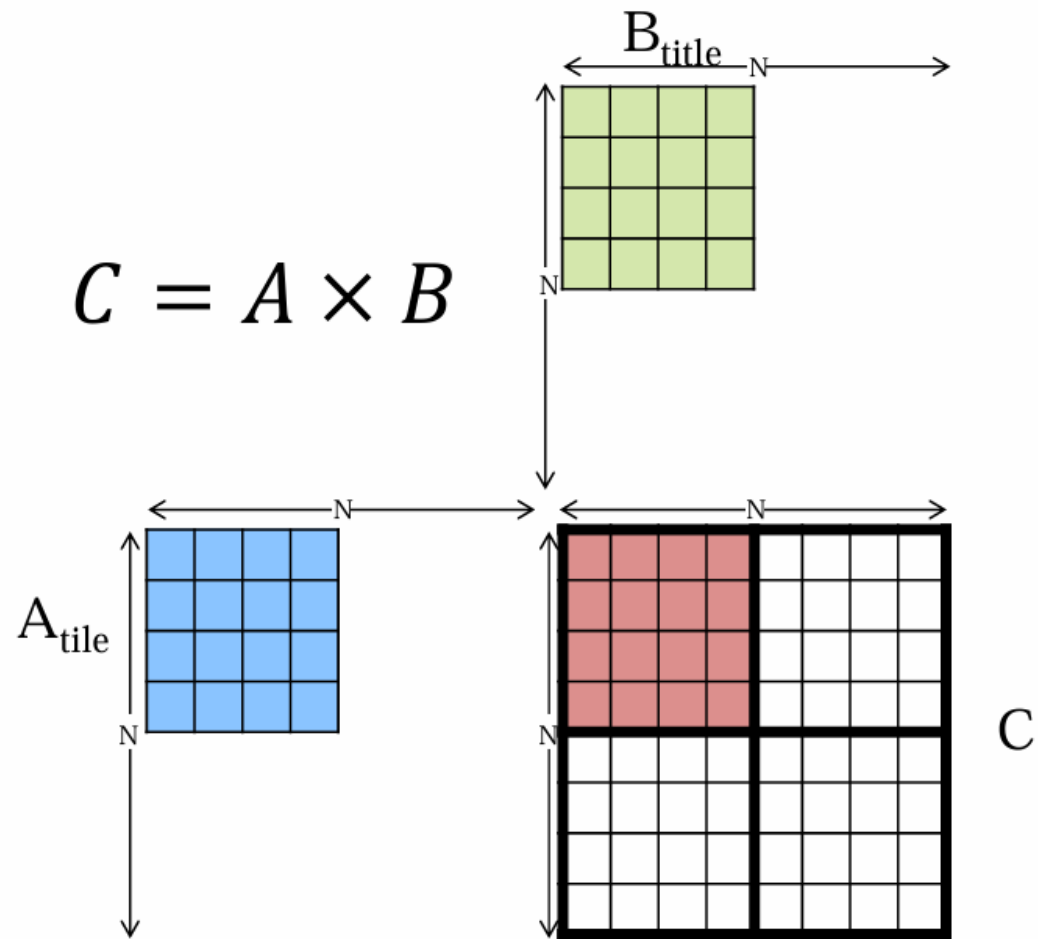
Step 1 (simultaneously)

B load the first tile of each input matrix to shared memory

each thread in the thread block loads one element

C wait for loading `__syncthreads()`

Tiling for Matrix Multiplication

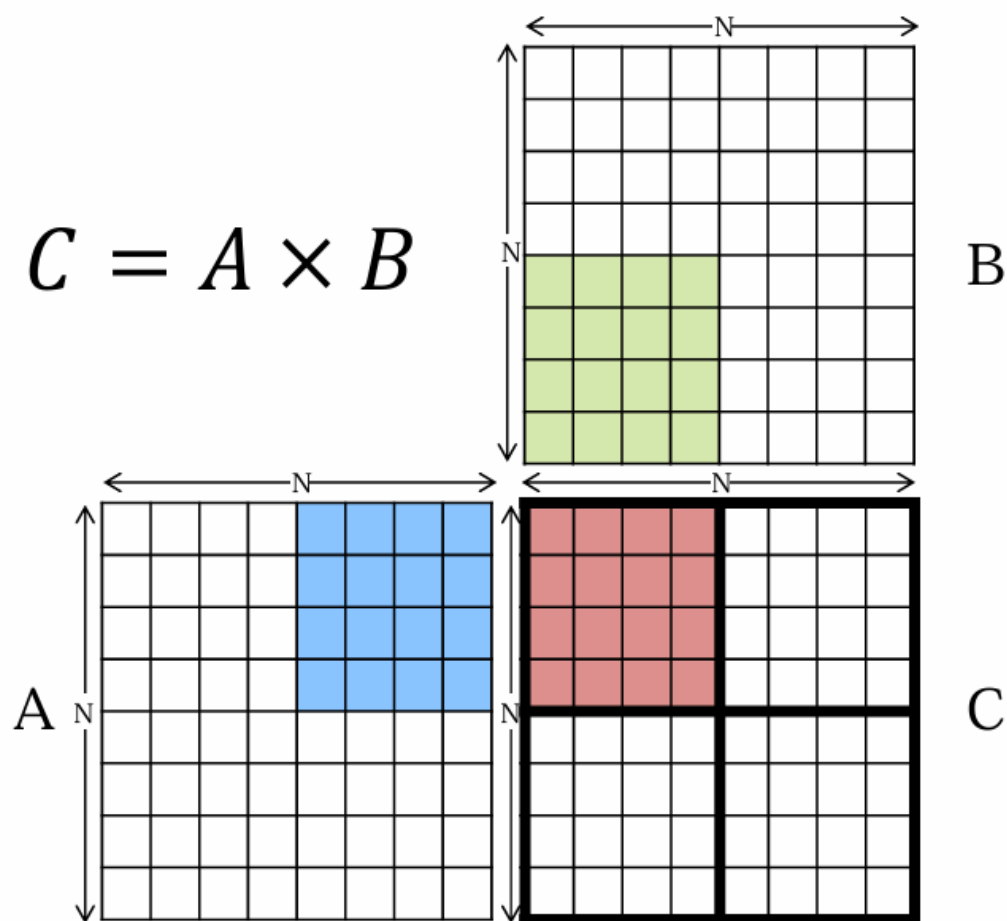


Step 2

each thread in the thread block
computes the partial sum from the
tiles in shared memory, threads wait
for each other to finish

`__syncthreads()`

Tiling for Matrix Multiplication



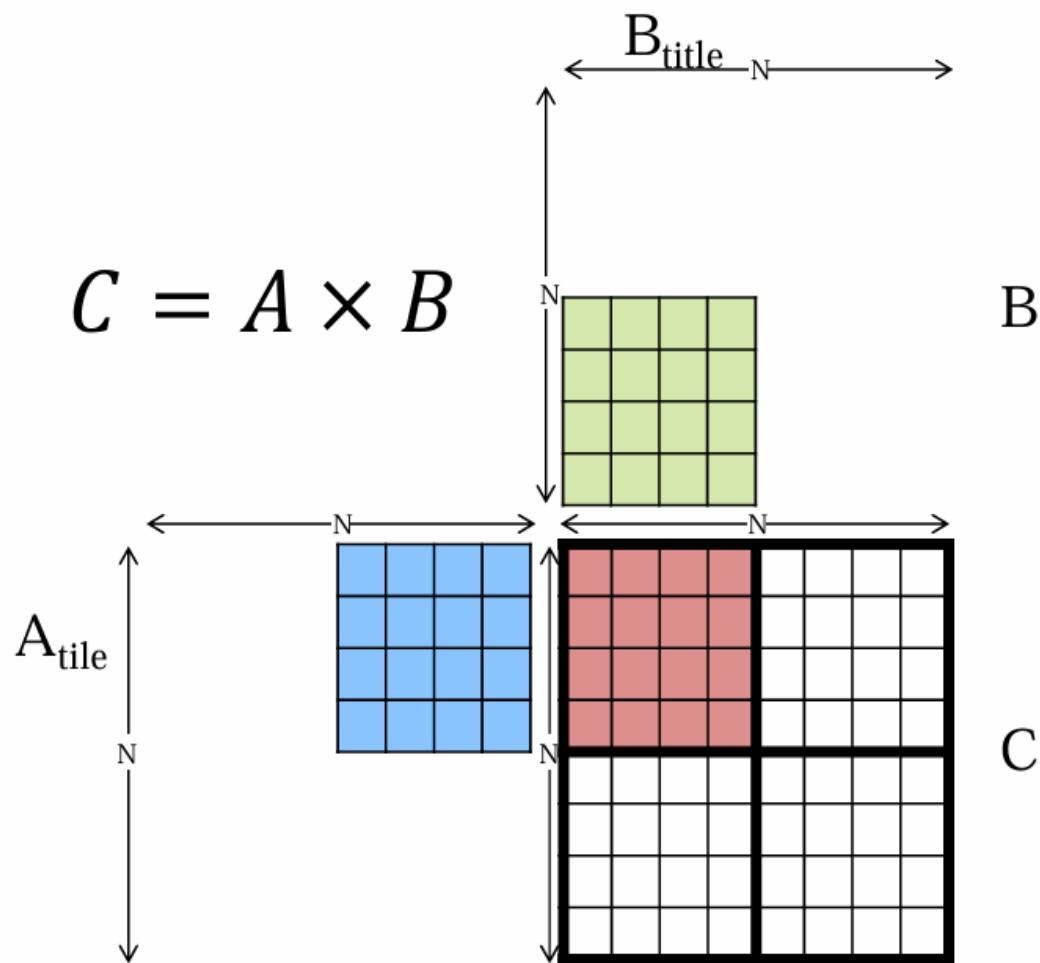
Step 3 (simultaneously)

B load the next tile of each input matrix to shared memory

each thread in the thread block loads one element

C

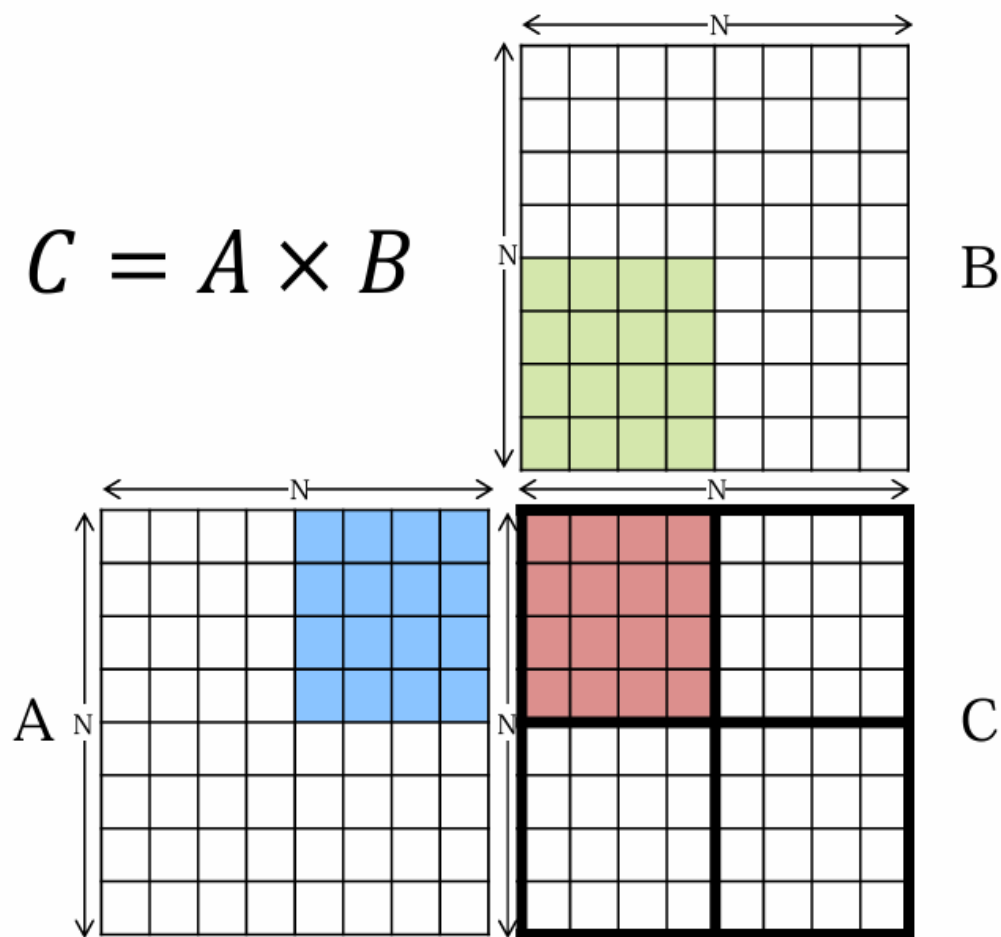
Tiling for Matrix Multiplication



Step 4

- B each thread in the thread block updates the partial sum from the tiles in shared memory, threads wait for each other to finish

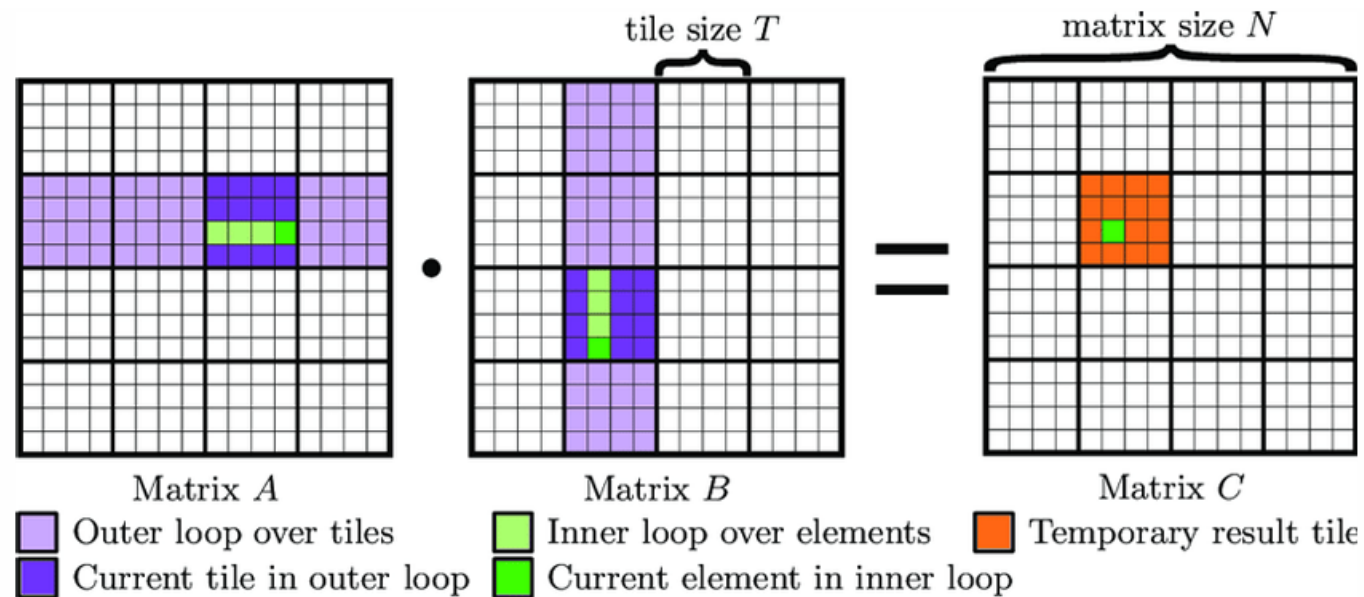
Tiling for Matrix Multiplication



Step 5, 6, 7, 8 ...

continue for next tiles

Tiling vs Standard



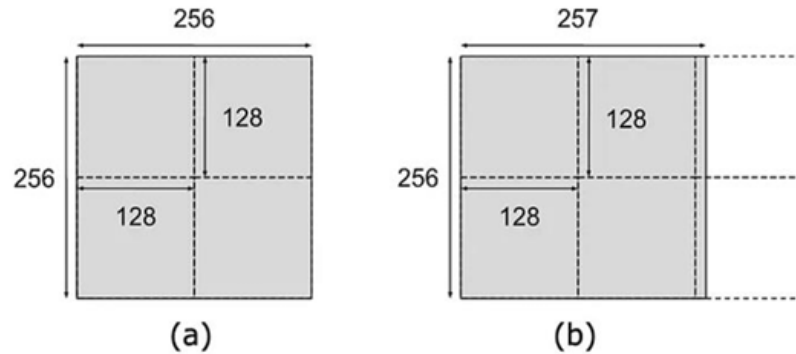
Non-tiled matrix multiply: each input is read N times from global memory

Tiled matrix multiply: each input is read $\frac{N}{T}$ times from global memory, and T times within each tile. This is a factor of T reduction in global memory access

Complexities with Tiling

Tile sizes may not divide the matrix size and lead to low utilization

Figure 6. Example of tiling with 128x128 thread block tiles. (a) Best case - matrix dimensions are divisible by tile dimensions (b) Worse case - tile quantization results in six thread blocks being launched, two of which waste most of their work.

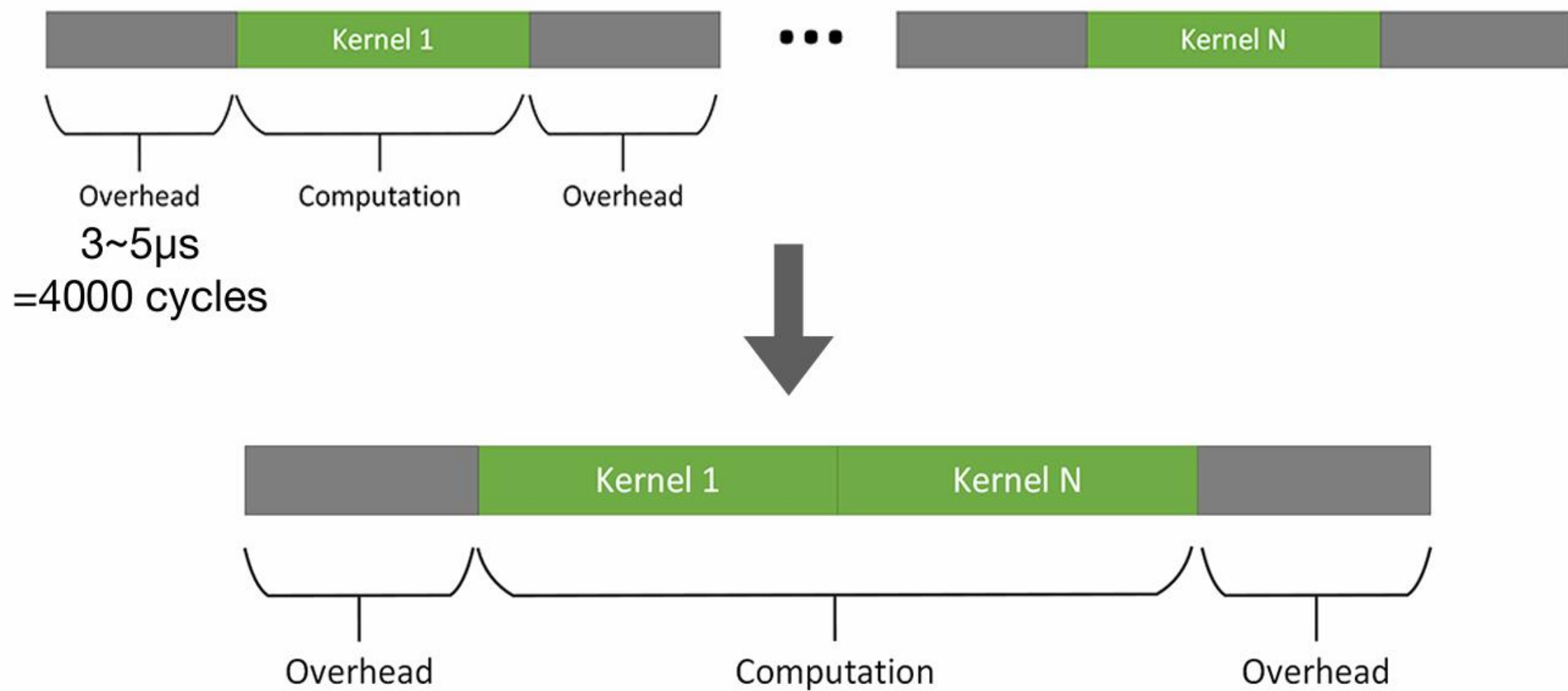


Factors affecting tile sizes

- Coalesced memory access
- Shared memory size
- Divisibility of the matrix dim

GPU Acceleration

- Tiling
- Kernel Fusion
- Low precision computation
- Recomputation



Kernel Fusion Example

$$C = A + B$$

4 load, 2 stores, 2 matrix add.

$$E = C + D$$

needs two kernel executions.

If we write a custom kernel to add three matrices

$$E = A + B + C$$

3 load, 1 store, 2 matrix add.

only needs one kernel.

Benefits:

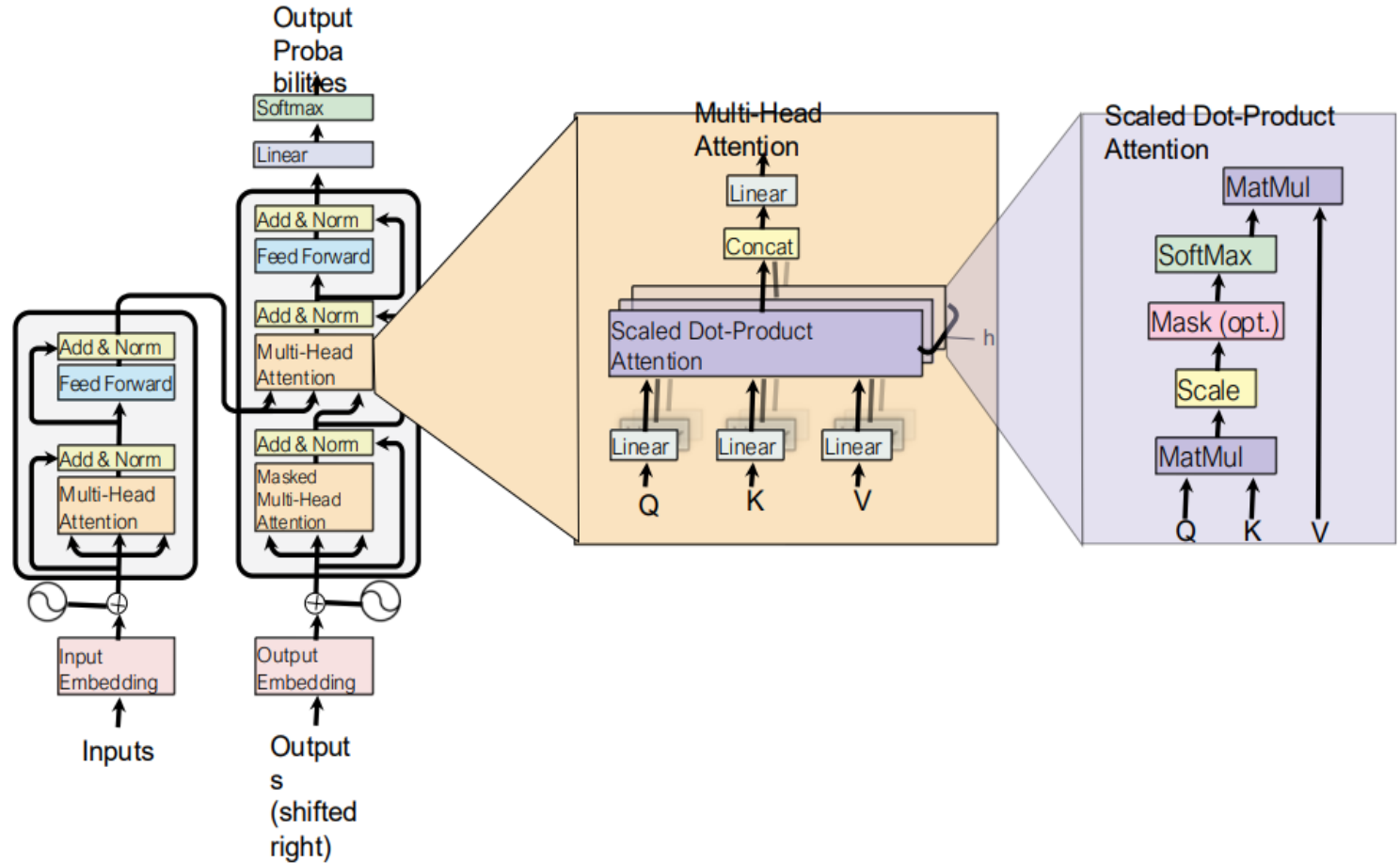
- reduce overhead
- reduce extra memory access

Outline

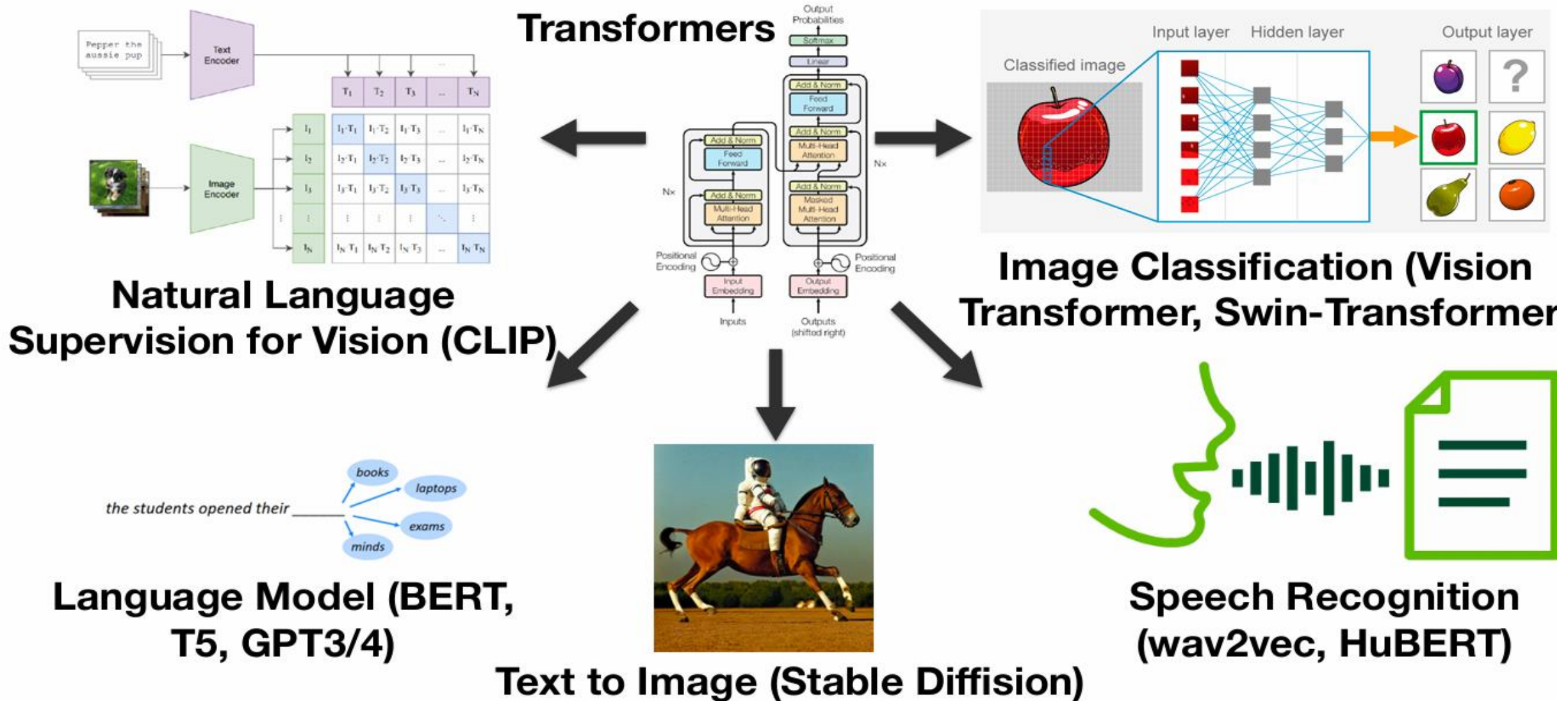
- GPU Architecture
- GPU Programming
- GPU Bottlenecks
- GPU Acceleration
- **LLM (Transformer)**
- GPU Acceleration for Transformer

Transformer in Original Paper

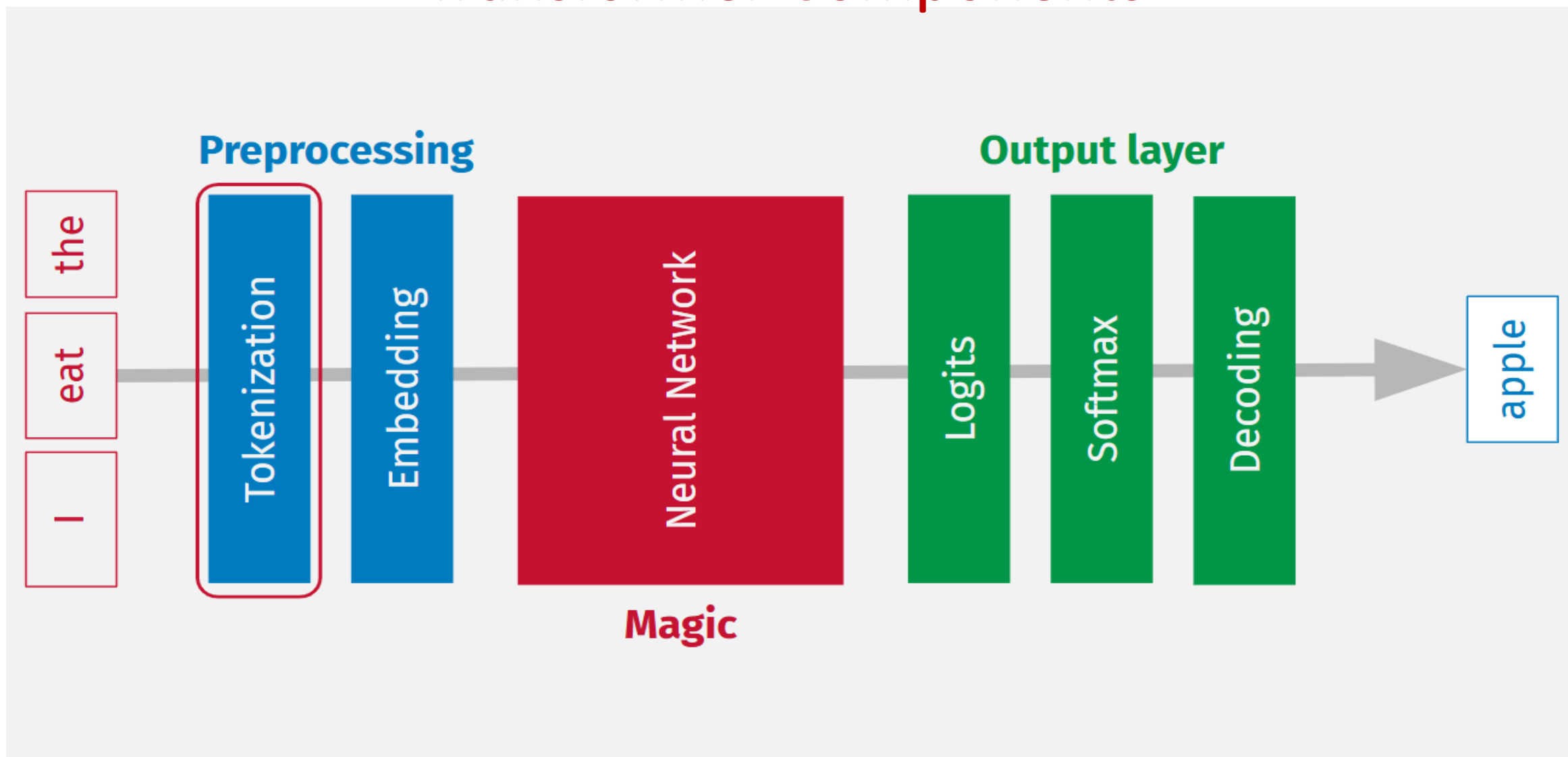
- C layers of encoder (=6)
- D layers of decoder (=6)
- Token Embedding: 512 (base), 1024 (large)
- FFN dim=2048



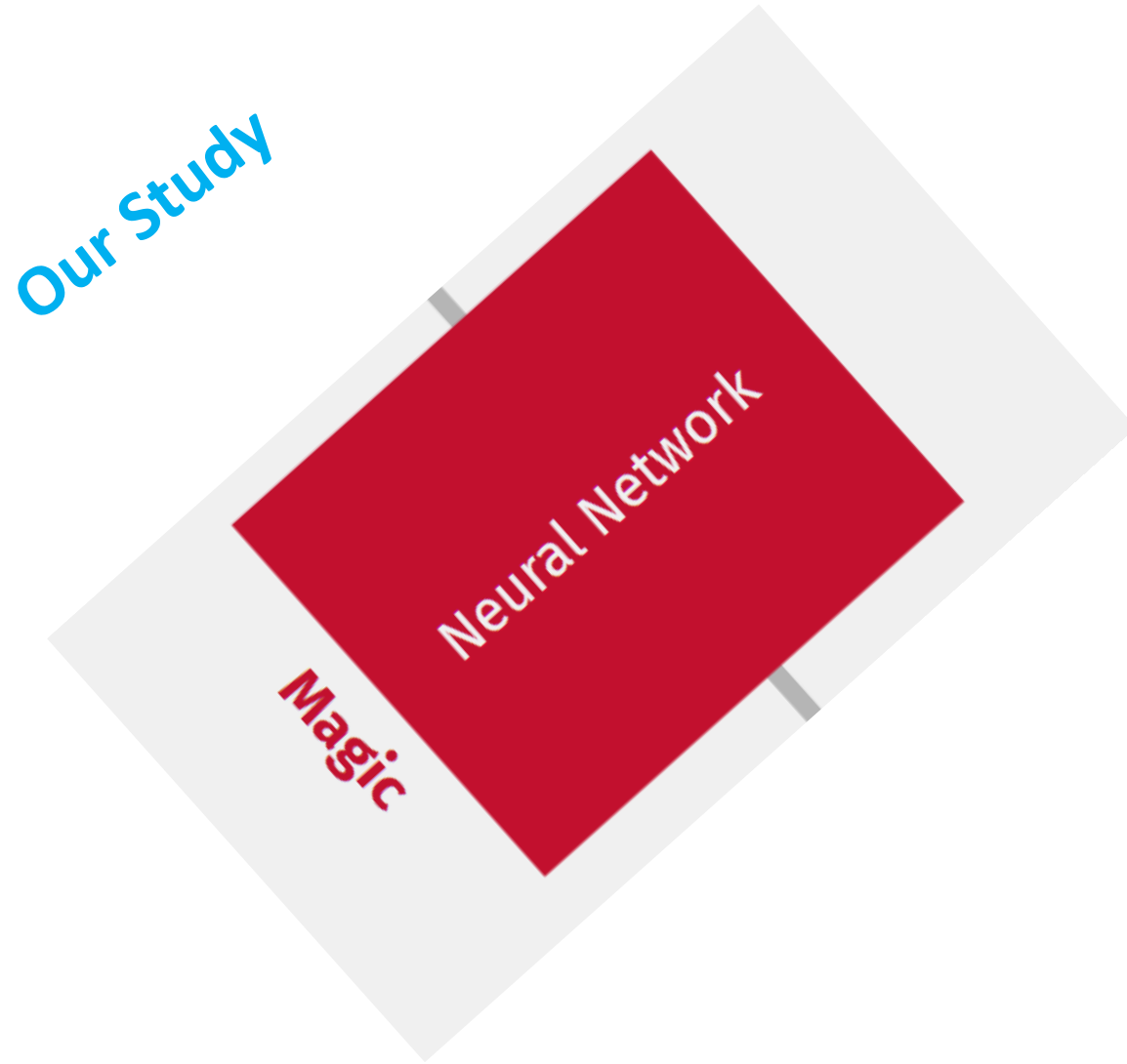
Transformer Models as universal architecture



Transformer Components



Transformer Components



Self Attention

h_1

↑
 x_1

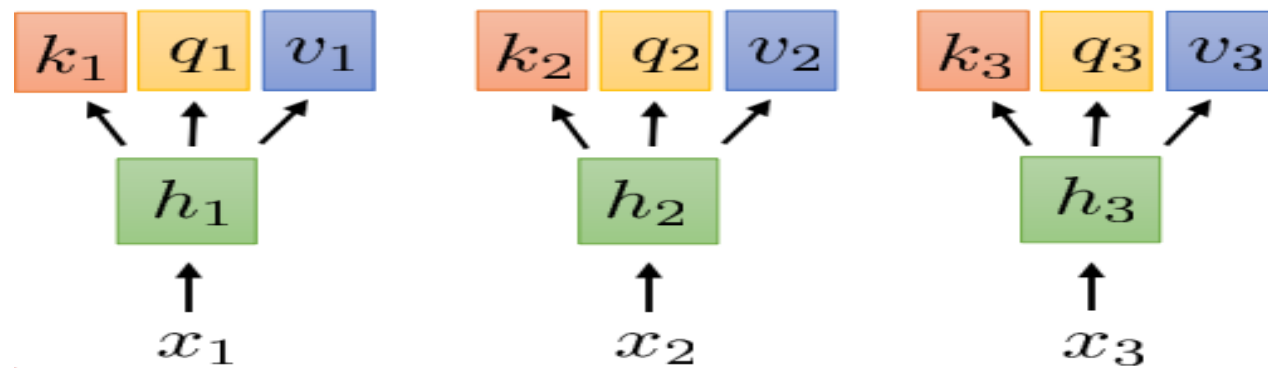
h_2

↑
 x_2

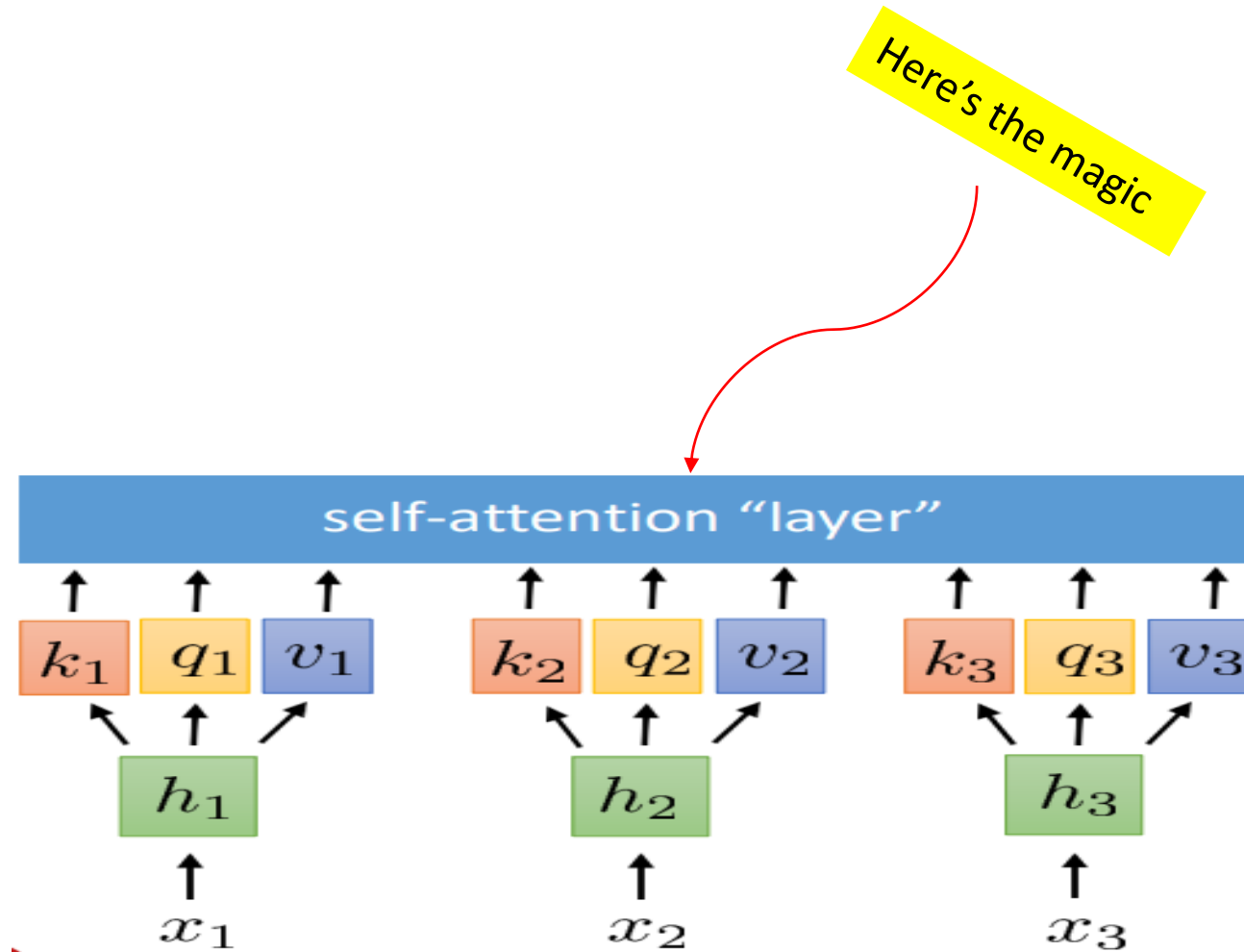
h_3

↑
 x_3

Self Attention



Self Attention

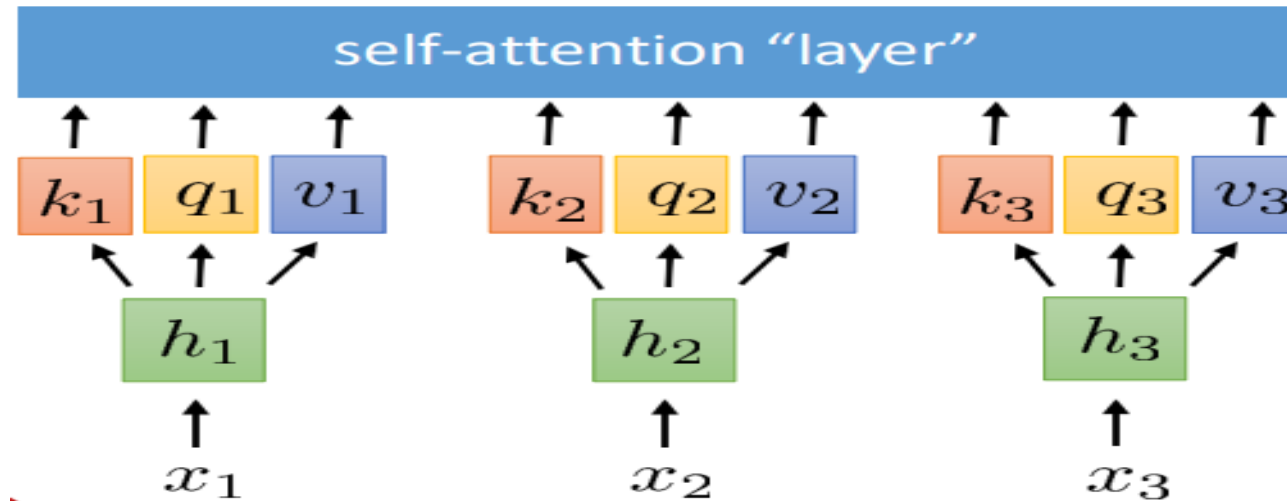


Self Attention

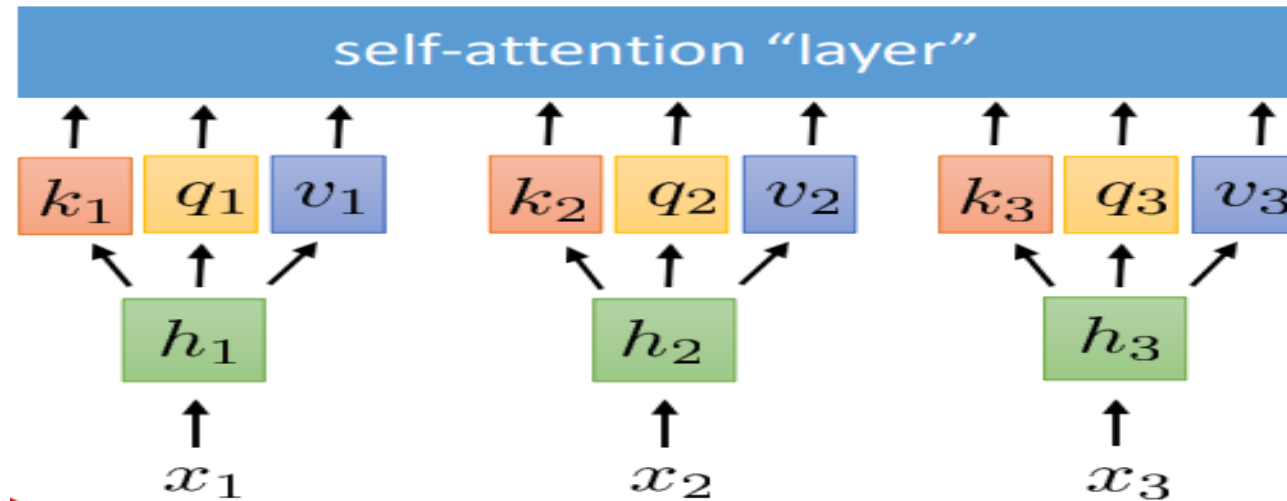
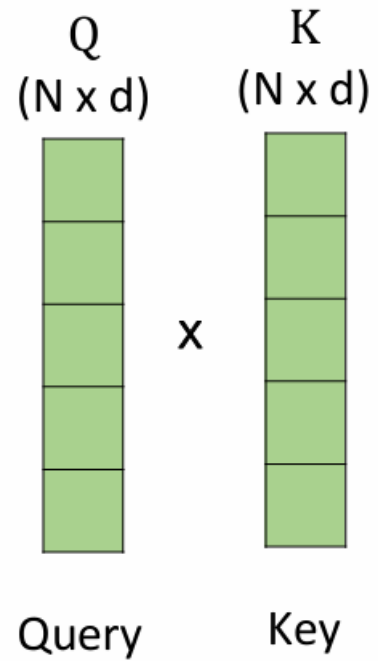
Q
(N x d)



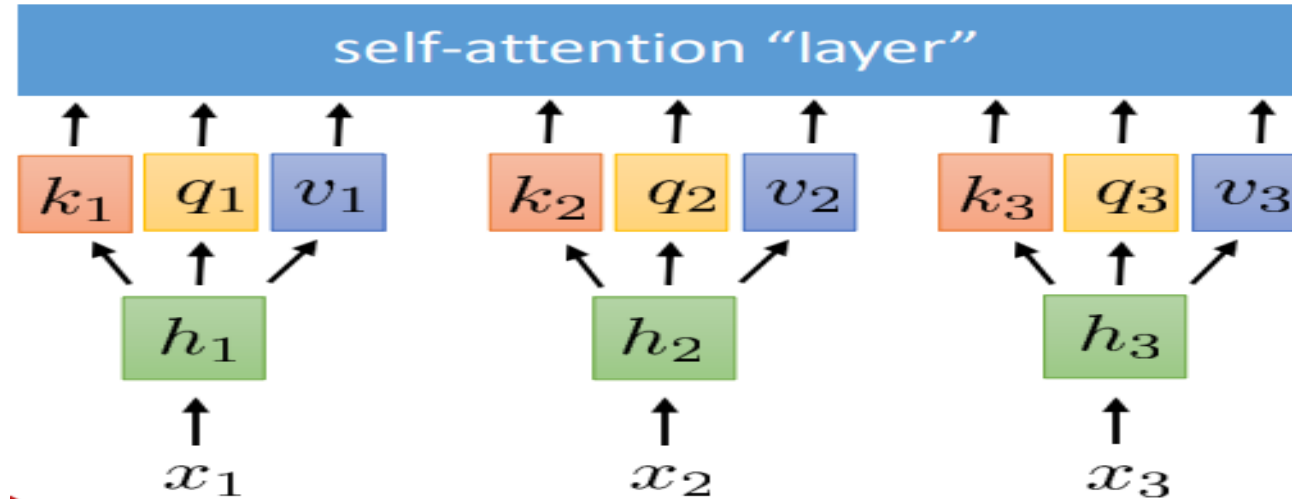
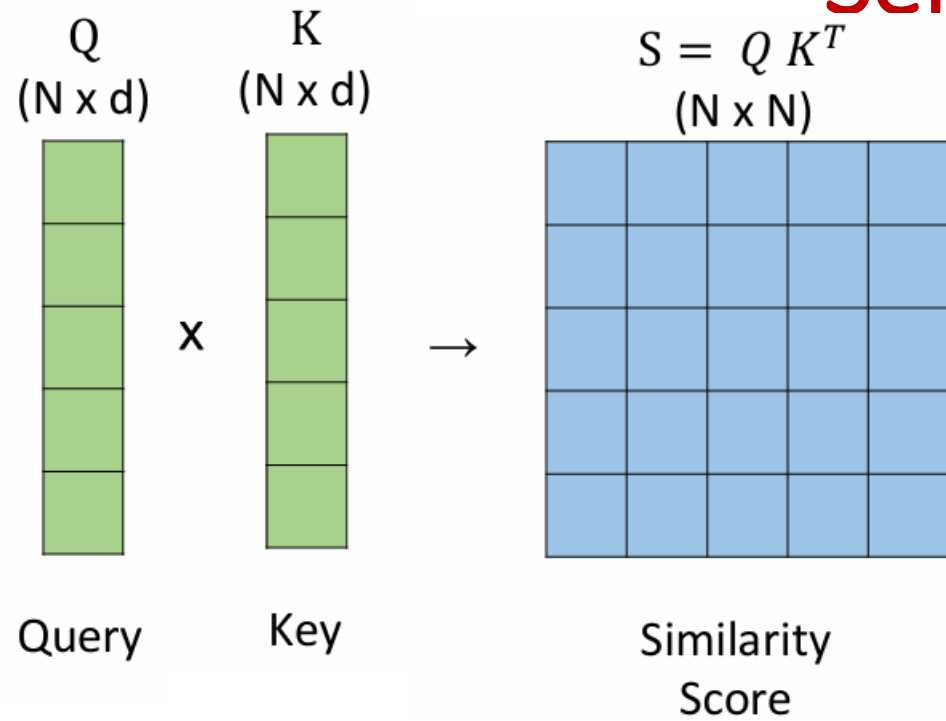
Query



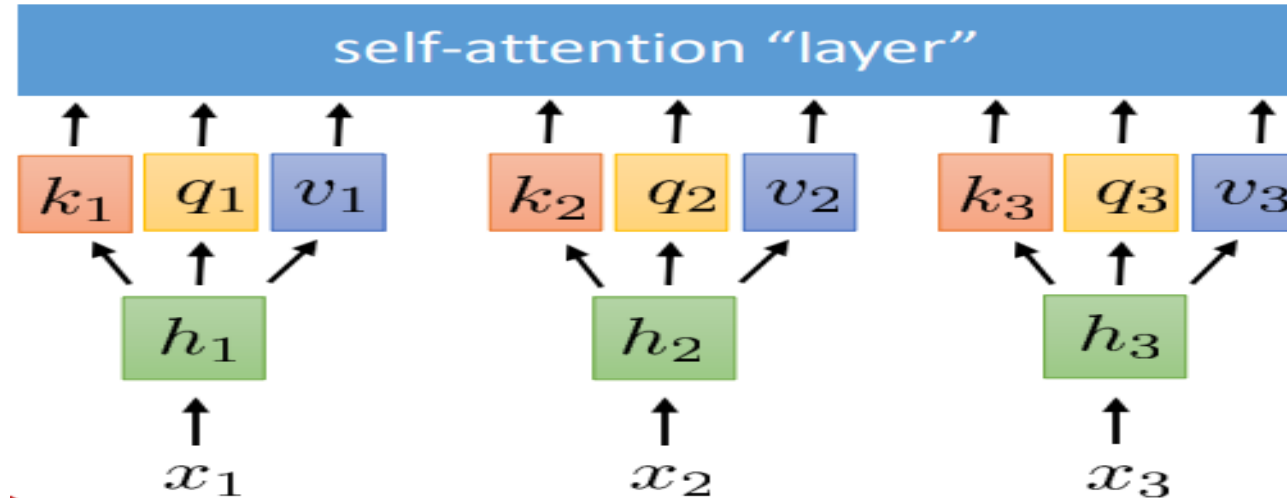
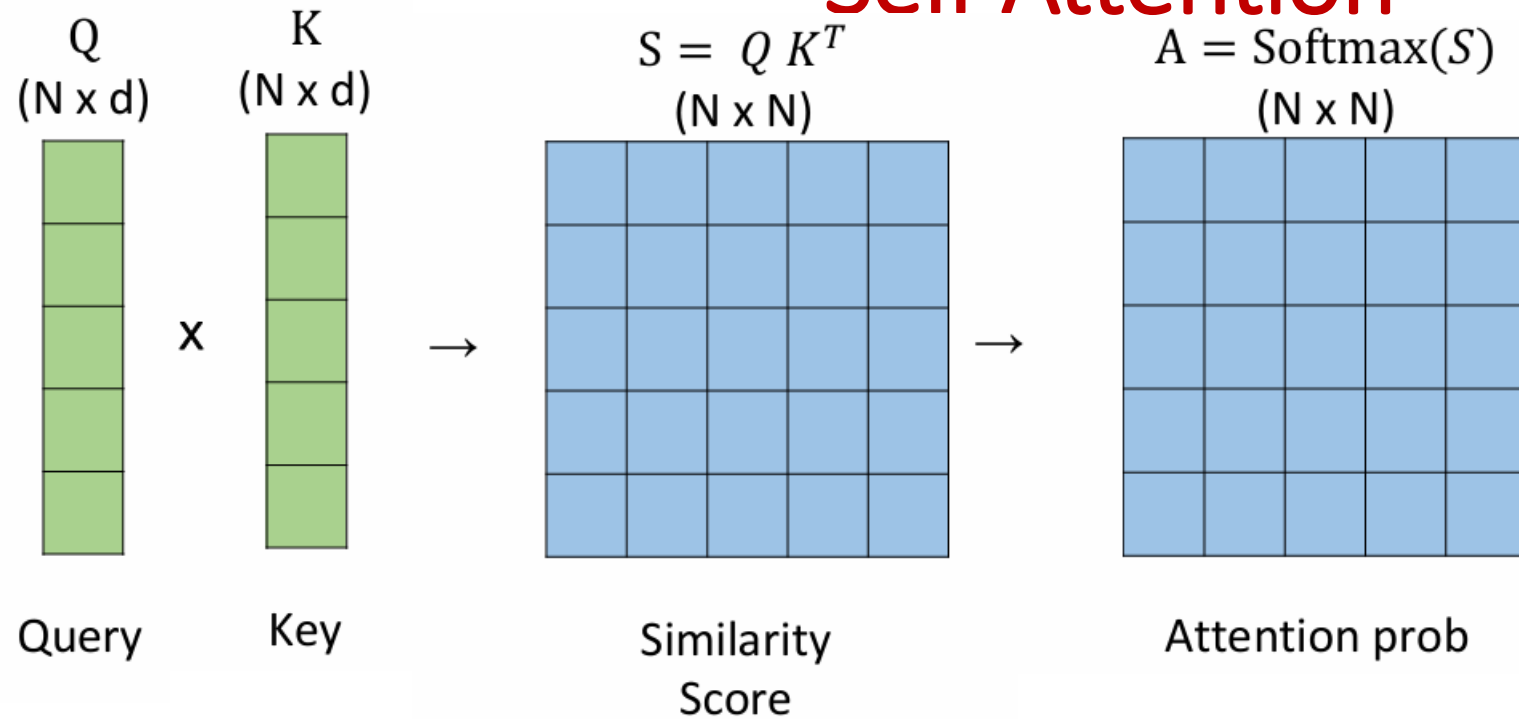
Self Attention



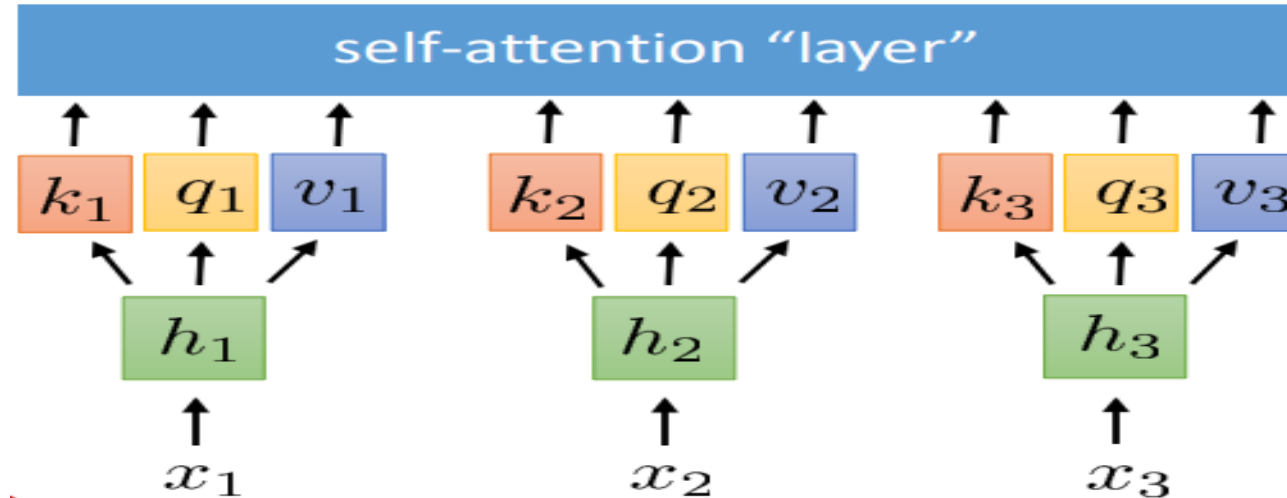
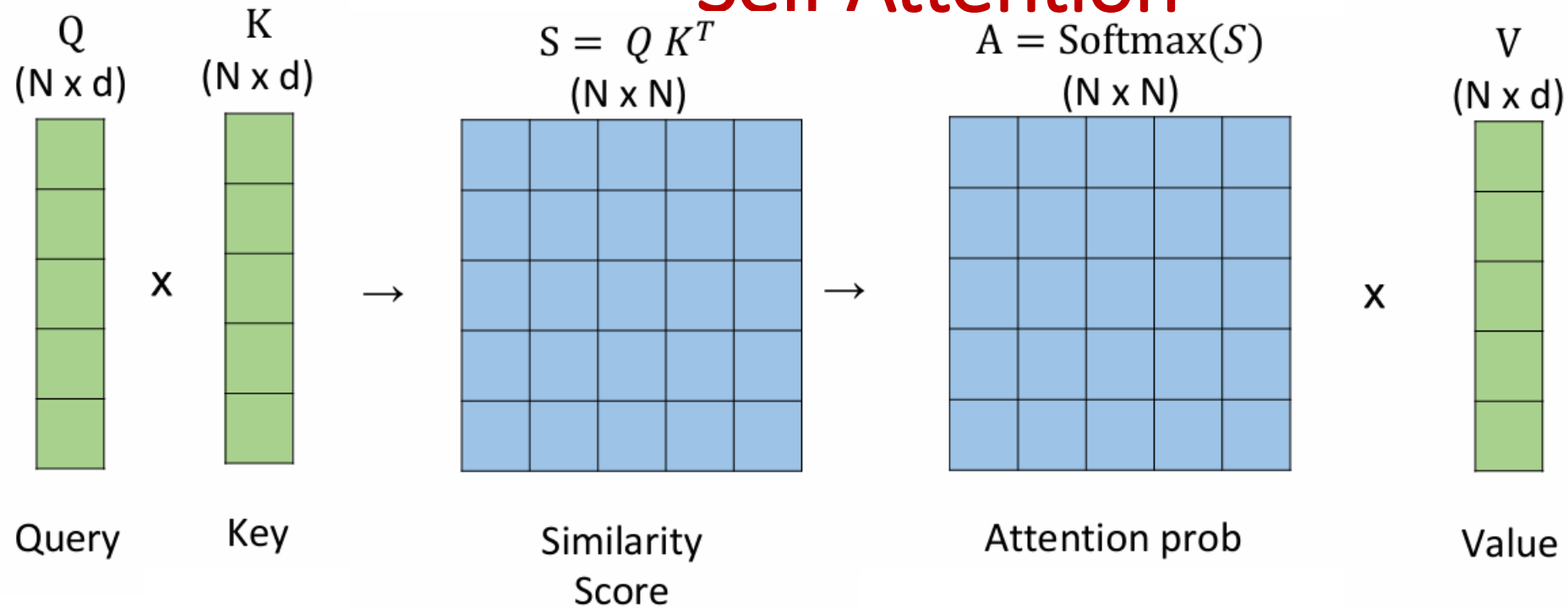
Self Attention



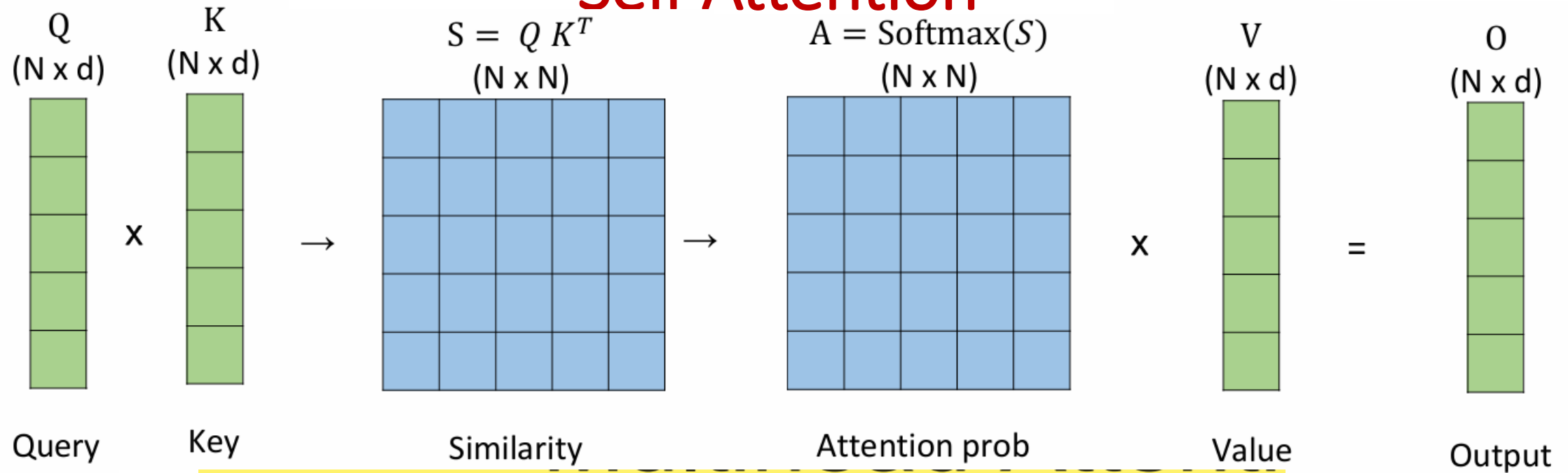
Self Attention



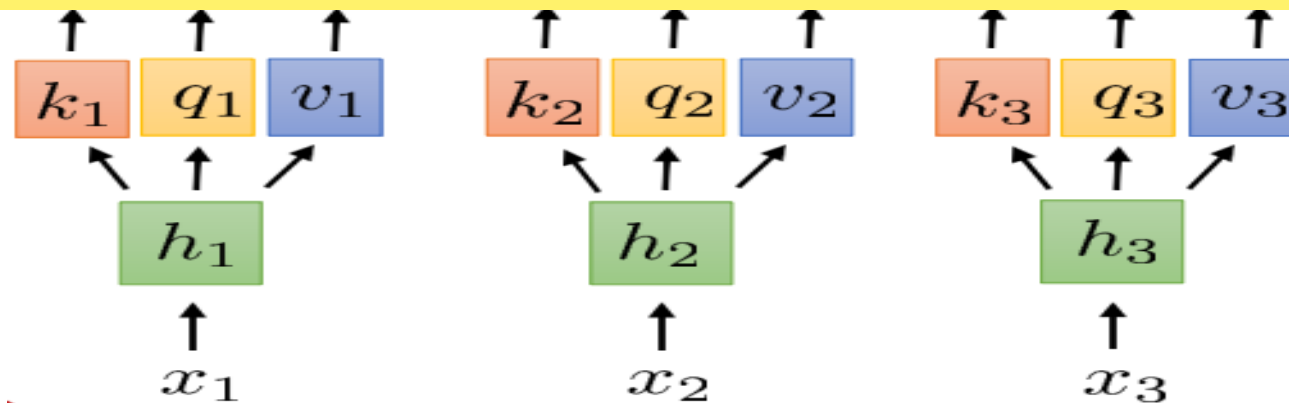
Self Attention



Self Attention



$$\text{Attention}(Q, K, V, x) = \text{Softmax}\left(\frac{(Qx)^T Kx}{\sqrt{d}}\right) \cdot (Vx)^T$$

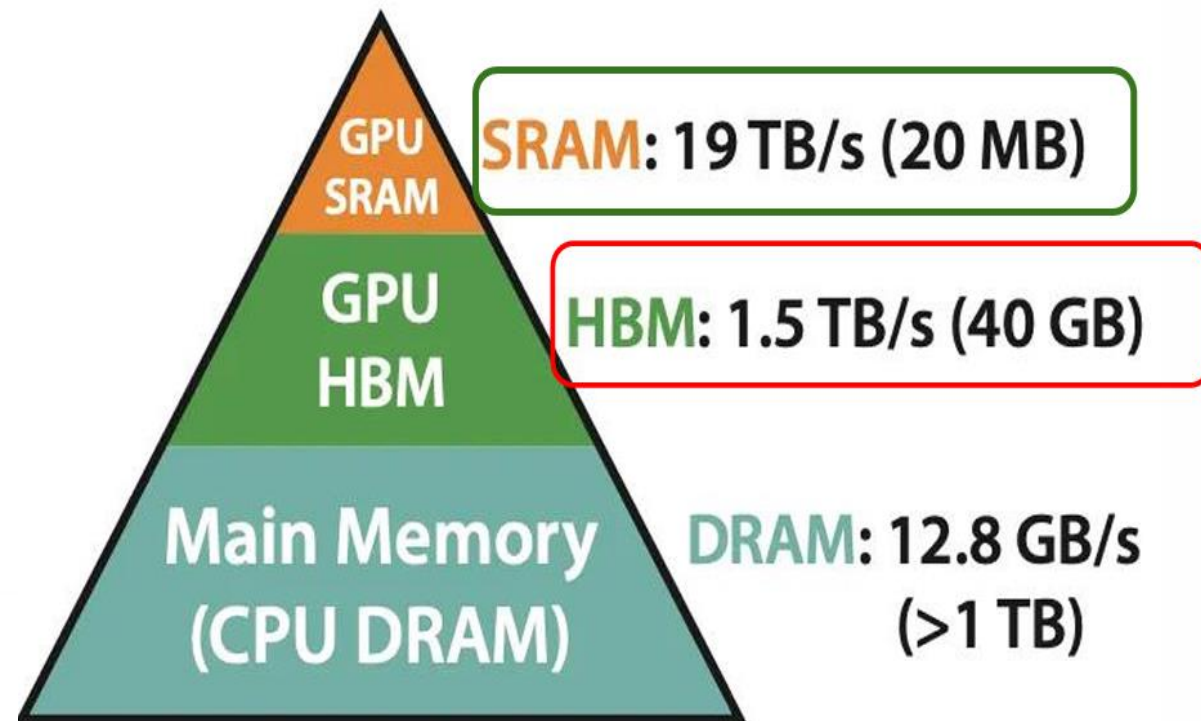


Attention On GPU

1. Matmul_op (Q,K)
 - a. Read Q,K to SRAM (read-op)
 - b. Compute matmul $A=Q \times K$ (compute-op)
 - c. Write A to HBM (write-op)
2. Mask_op
 - a. Read A to SRAM (read-op)
 - b. Mask A into A' (compute-op)
 - c. Write A' to HBM (write-op)
3. Softmax_op
 - a. Read A' to SRAM (read-op)
 - b. Softmax A' into A'' (compute-op)
 - c. Write A'' to HBM (write-op)

Attention On GPU

1. Matmul_op (Q,K)
 - a. Read Q,K to SRAM (read-op)
 - b. Compute matmul $A=Q \times K$ (compute-op)
 - c. Write A to HBM (write-op)
2. Mask_op
 - a. Read A to SRAM (read-op)
 - b. Mask A into A' (compute-op)
 - c. Write A' to HBM (write-op)
3. Softmax_op
 - a. Read A' to SRAM (read-op)
 - b. Softmax A' into A'' (compute-op)
 - c. Write A'' to HBM (write-op)

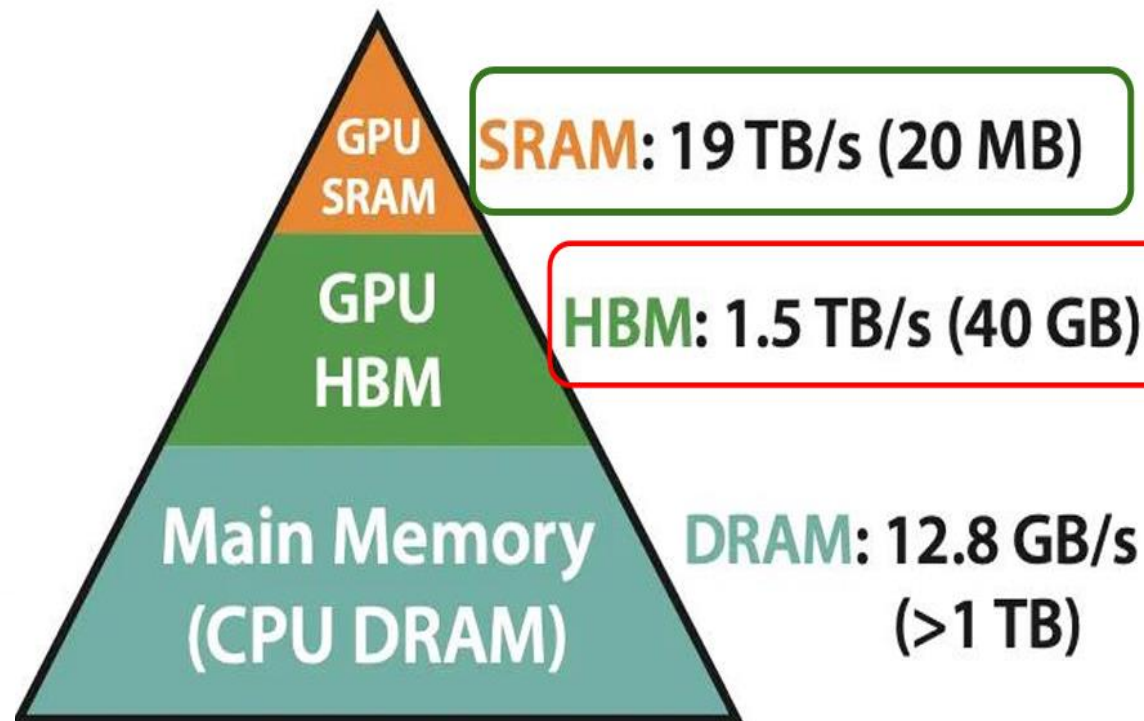


Attention On GPU

*How to Reduce HBM
Reads/Writes*

1. Matmul_op (Q,K)
 - a. Read Q,K to SRAM (read-op)
 - b. Compute matmul $A=Q \times K$ (compute-op)
 - c. Write A to HBM (write-op)
2. Mask_op
 - a. Read A to SRAM (read-op)
 - b. Mask A into A' (compute-op)
 - c. Write A' to HBM (write-op)
3. Softmax_op
 - a. Read A' to SRAM (read-op)
 - b. Softmax A' into A'' (compute-op)
 - c. Write A'' to HBM (write-op)

*Attention is Bottlenecked
by Memory Reads/Writes*

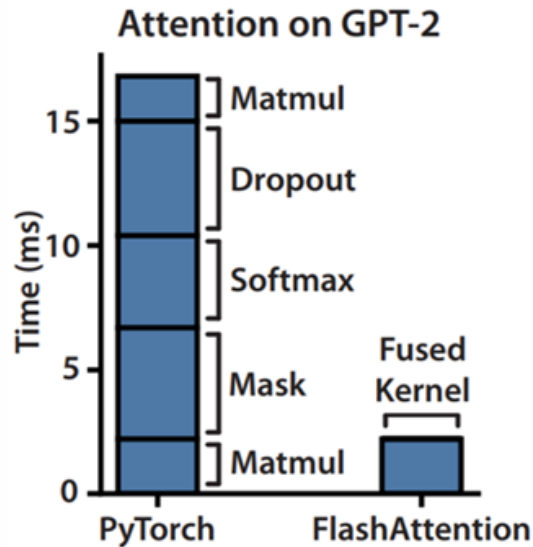


Outline

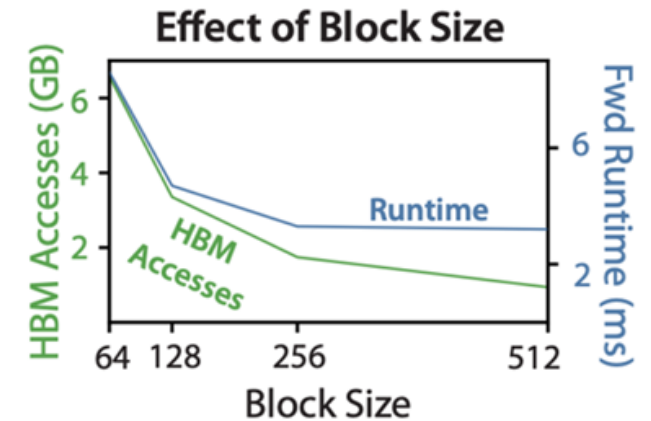
- GPU Architecture
- GPU Programming
- GPU Bottlenecks
- GPU Acceleration
- LLM (Transformer)
- GPU Acceleration for Transformer (**Flash Attention**)

Using what we know to understand Flash Attention

Flash attention [Dao et al] dramatically accelerates attention.. But how?



Attention	Standard	FLASHATTENTION
GFLOPs	66.6	75.2
HBM R/W (GB)	40.3	4.4
Runtime (ms)	41.7	7.3



Technique from paper:

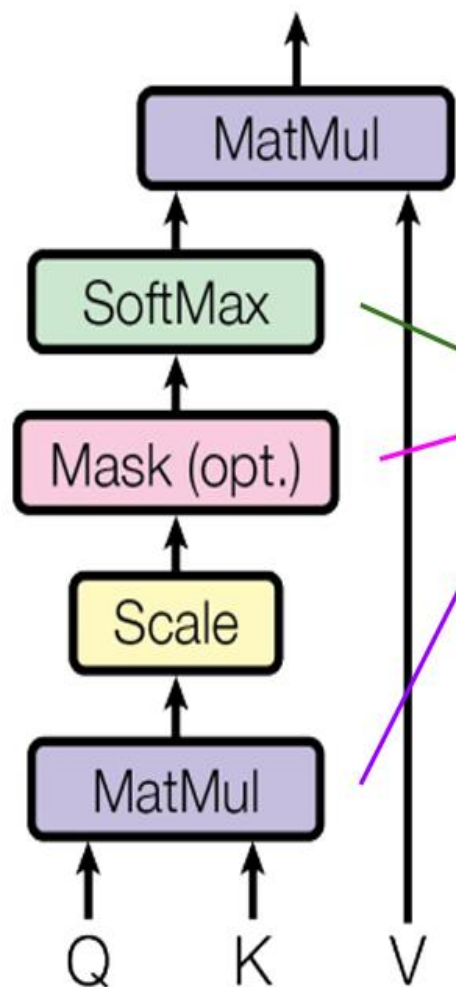
We apply two established techniques (tiling, recomputation) to overcome the technical challenge of computing exact attention in sub-quadratic HBM accesses. We describe this in Algorithm 1. The main idea

Key Takeaways

- Avoid unnecessary HBM read/write
- Maximize SRAM computation



The magic: Fused Kernel (GPU Operations)!

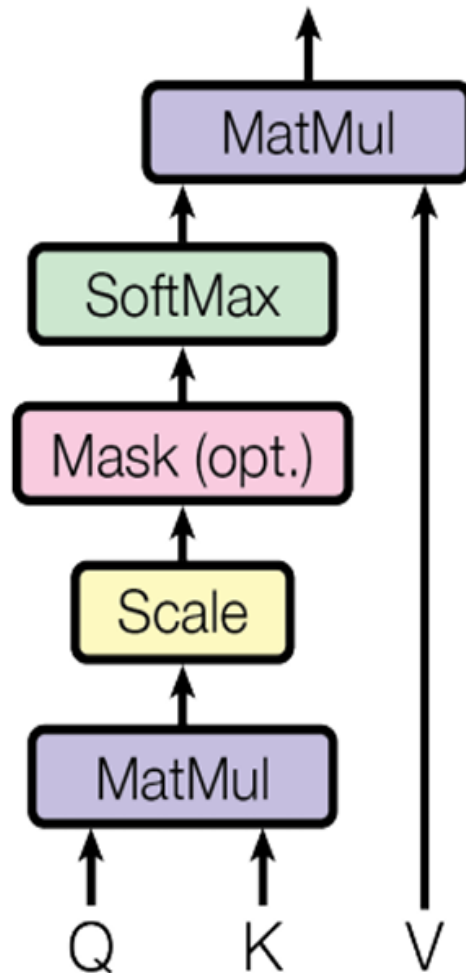


Flash Attention

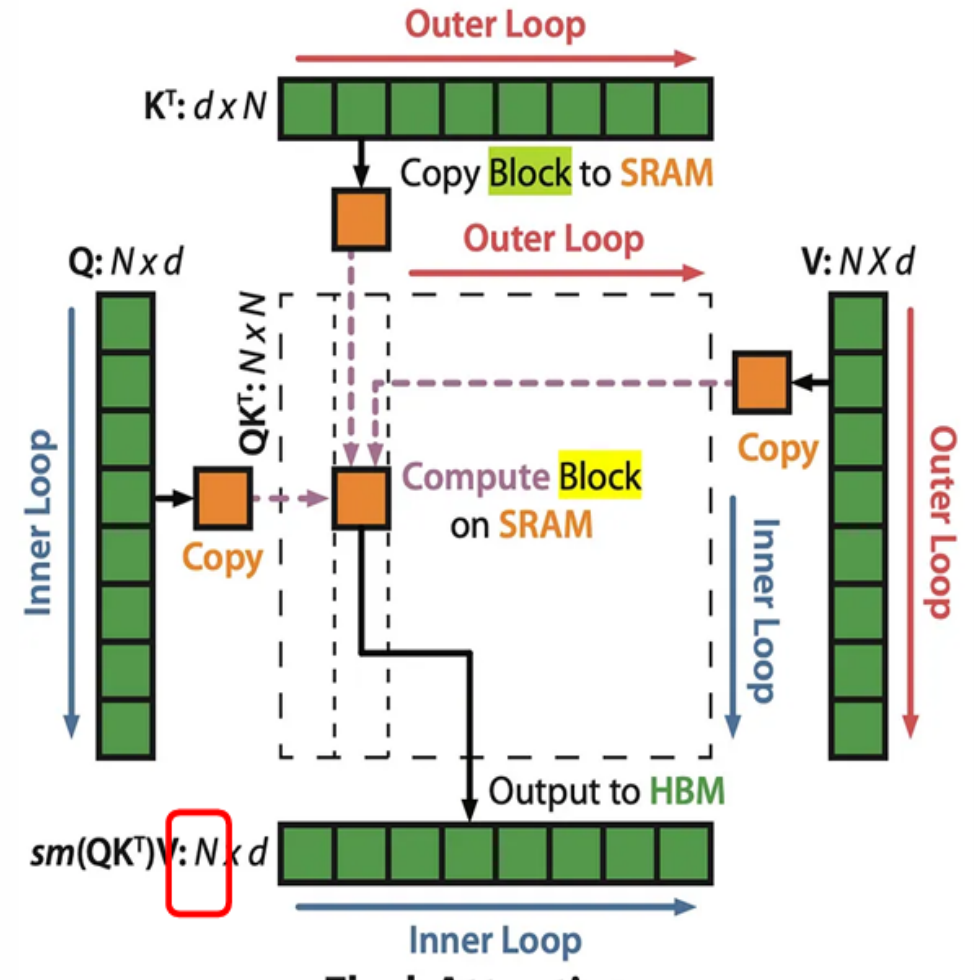
1. Read Q,K to SRAM
2. Compute $A = Q \times K$
3. Mask A into A'
4. Softmax A' into A''
5. Write A'' to HBM

1. Matmul_op (Q,K)
 - a. Read Q,K to SRAM (read-op)
 - b. Compute matmul $A=Q \times K$ (compute-op)
 - c. Write A to HBM (write-op)
2. Mask_op
 - a. Read A to SRAM (read-op)
 - b. Mask A into A' (compute-op)
 - c. Write A' to HBM (write-op)
3. Softmax_op
 - a. Read A' to SRAM (read-op)
 - b. Softmax A' into A'' (compute-op)
 - c. Write A'' to HBM (write-op)

Does the story end here? What's the problem?



$$s(x_i) = \frac{e^{x_i}}{\sum_{j=1}^N e^{x_j}}$$



The softmax denominator problem

From Mikailov and Gimelshein 2018,

Normal softmax

$$y_i = \frac{e^{x_i - \max_{k=1}^V x_k}}{\sum_{j=1}^V e^{x_j - \max_{k=1}^V x_k}} \quad (2)$$

All major DL frameworks are using this safe version for the Softmax computation: TensorFlow

Algorithm 2 Safe softmax

```
1:  $m_0 \leftarrow -\infty$ 
2: for  $k \leftarrow 1, V$  do
3:    $m_k \leftarrow \max(m_{k-1}, x_k)$ 
4: end for
5:  $d_0 \leftarrow 0$ 
6: for  $j \leftarrow 1, V$  do
7:    $d_j \leftarrow d_{j-1} + e^{x_j - m_V}$ 
8: end for
9: for  $i \leftarrow 1, V$  do
10:   $y_i \leftarrow \frac{e^{x_i - m_V}}{d_V}$ 
11: end for
```

Online softmax

Algorithm 3 Safe softmax with online normalizer calculation

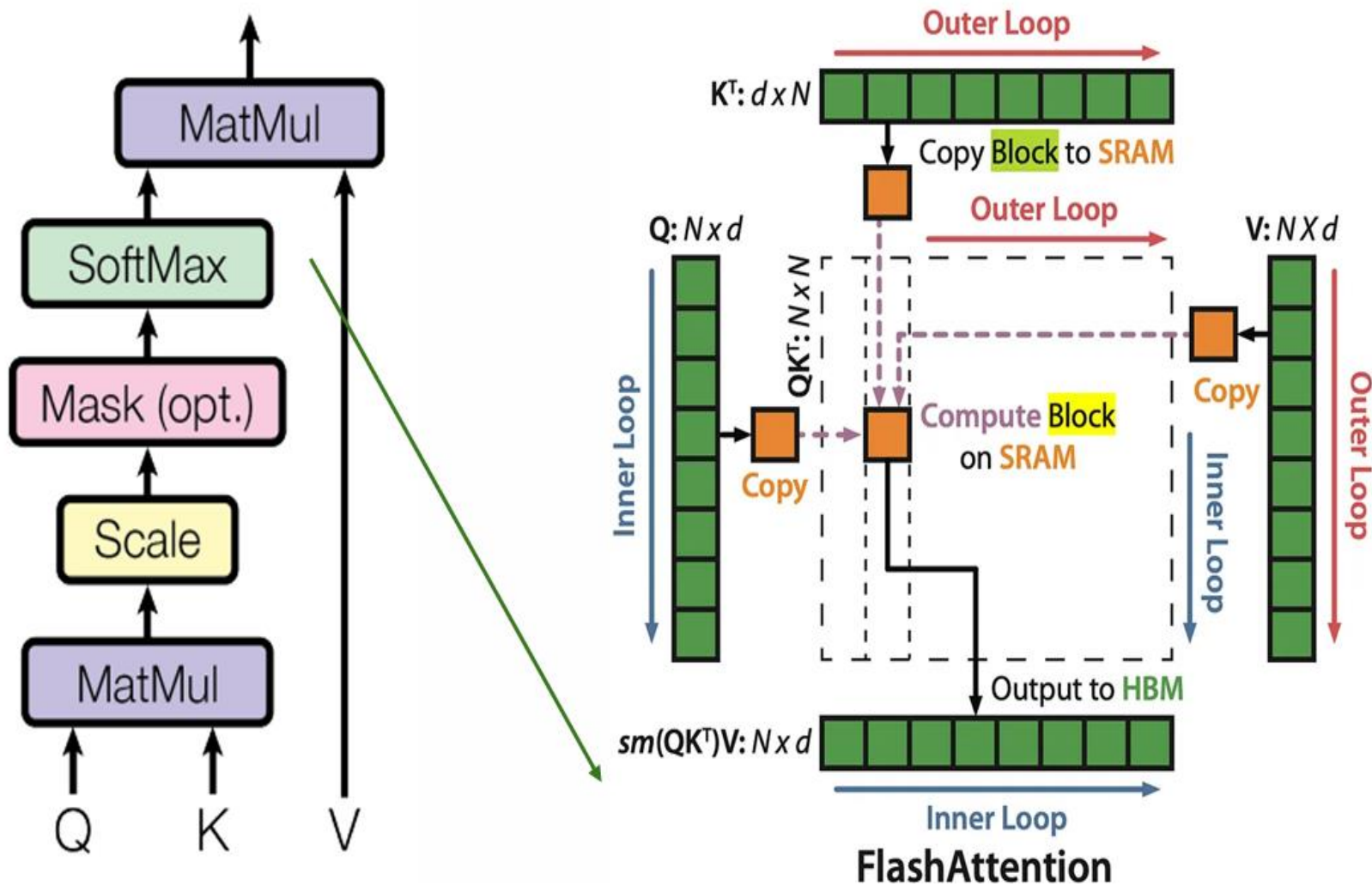
```
1:  $m_0 \leftarrow -\infty$ 
2:  $d_0 \leftarrow 0$ 
3: for  $j \leftarrow 1, V$  do
4:    $m_j \leftarrow \max(m_{j-1}, x_j)$ 
5:    $d_j \leftarrow d_{j-1} \times e^{m_{j-1} - m_j} + e^{x_j - m_j}$ 
6: end for
7: for  $i \leftarrow 1, V$  do
8:    $y_i \leftarrow \frac{e^{x_i - m_V}}{d_V}$ 
9: end for
```

To keep track of the max, incrementally update the max, and set up a telescoping sum

This lets you compute the softmax tile-by-tile



The magic does not end here! Tiling



Year: 2022; NeurIPS



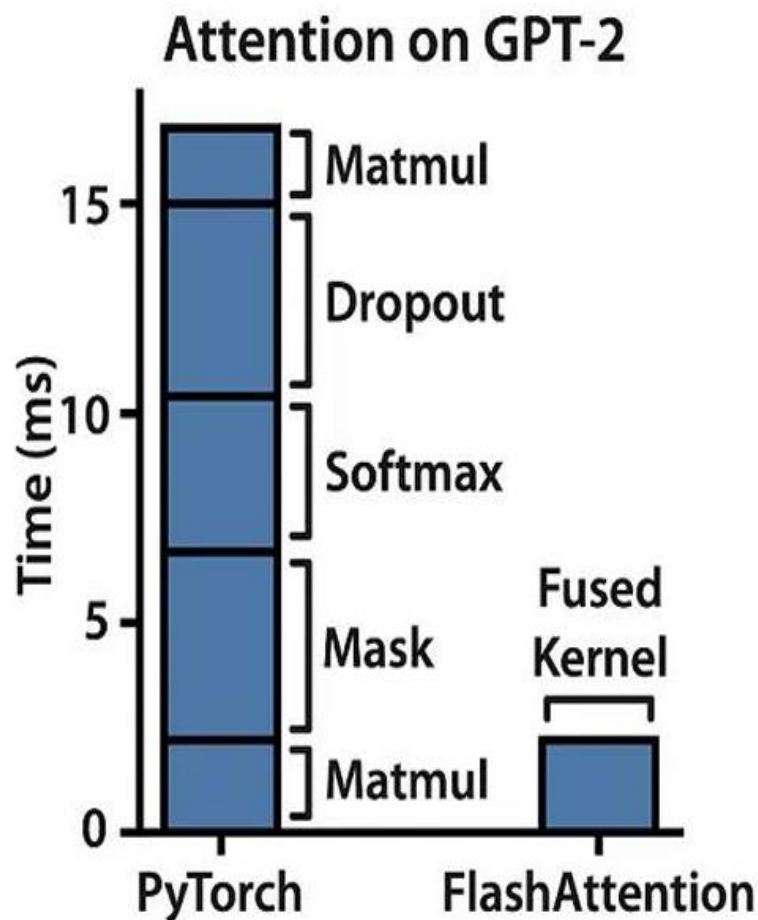
The magic does not end here!

Recomputation activation During backward

Year: 2022; NeurIPS



How well did they do?



BERT Implementation	Training time (minutes)
Nvidia MLPerf 1.1 [58]	20.0 ± 1.5
FLASHATTENTION (ours)	17.4 ± 1.4

*Time: $O(N*d)$*
*Space: $O(N*d)$*



FlashAttention2 Paper

Forward Pass

Algorithm 1 FLASHATTENTION-2 forward pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, block sizes B_c, B_r .

- 1: Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
 - 2: Divide the output $\mathbf{O} \in \mathbb{R}^{N \times d}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, and divide the logsumexp L into T_r blocks $L_i, \dots, L_{T_r}$ of size B_r each.
 - 3: **for** $1 \leq i \leq T_r$ **do**
 - 4: Load $\mathbf{Q}_i$ from HBM to on-chip SRAM.
 - 5: On chip, initialize $\mathbf{O}_i^{(0)} = (0)_{B_r \times d} \in \mathbb{R}^{B_r \times d}, \ell_i^{(0)} = (0)_{B_r} \in \mathbb{R}^{B_r}, m_i^{(0)} = (-\infty)_{B_r} \in \mathbb{R}^{B_r}$.
 - 6: **for** $1 \leq j \leq T_c$ **do**
 - 7: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
 - 8: On chip, compute $\mathbf{S}_i^{(j)} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
 - 9: On chip, compute $m_i^{(j)} = \max(m_i^{(j-1)}, \text{rowmax}(\mathbf{S}_i^{(j)})) \in \mathbb{R}^{B_r}, \tilde{\mathbf{P}}_i^{(j)} = \exp(\mathbf{S}_i^{(j)} - m_i^{(j)}) \in \mathbb{R}^{B_r \times B_c}$
(pointwise), $\ell_i^{(j)} = e^{m_i^{(j-1)} - m_i^{(j)}} \ell_i^{(j-1)} + \text{rowsum}(\tilde{\mathbf{P}}_i^{(j)}) \in \mathbb{R}^{B_r}$.
 - 10: On chip, compute $\mathbf{O}_i^{(j)} = \text{diag}(e^{m_i^{(j-1)} - m_i^{(j)}})^{-1} \mathbf{O}_i^{(j-1)} + \tilde{\mathbf{P}}_i^{(j)} \mathbf{V}_j$.
 - 11: **end for**
 - 12: On chip, compute $\mathbf{O}_i = \text{diag}(\ell_i^{(T_c)})^{-1} \mathbf{O}_i^{(T_c)}$.
 - 13: On chip, compute $L_i = m_i^{(T_c)} + \log(\ell_i^{(T_c)})$.
 - 14: Write $\mathbf{O}_i$ to HBM as the i -th block of $\mathbf{O}$.
 - 15: Write L_i to HBM as the i -th block of L .
 - 16: **end for**
 - 17: Return the output $\mathbf{O}$ and the logsumexp L .
-

Backward pass

Algorithm 2 FLASHATTENTION-2 Backward Pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}, \mathbf{dO} \in \mathbb{R}^{N \times d}$ in HBM, vector $L \in \mathbb{R}^N$ in HBM, block sizes B_c, B_r .

- 1: Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
- 2: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide $\mathbf{dO}$ into T_r blocks $\mathbf{dO}_1, \dots, \mathbf{dO}_{T_r}$ of size $B_r \times d$ each, and divide L into T_r blocks $L_1, \dots, L_{T_r}$ of size B_r each.
- 3: Initialize $\mathbf{dQ} = (0)_{N \times d}$ in HBM and divide it into T_r blocks $\mathbf{dQ}_1, \dots, \mathbf{dQ}_{T_r}$ of size $B_r \times d$ each. Divide $\mathbf{dK}, \mathbf{dV} \in \mathbb{R}^{N \times d}$ into T_c blocks $\mathbf{dK}_1, \dots, \mathbf{dK}_{T_c}$ and $\mathbf{dV}_1, \dots, \mathbf{dV}_{T_c}$, of size $B_c \times d$ each.
- 4: Compute $D = \text{rowsum}(\mathbf{dO} \circ \mathbf{O}) \in \mathbb{R}^d$ (pointwise multiply), write D to HBM and divide it into T_r blocks $D_1, \dots, D_{T_r}$ of size B_r each.
- 5: **for** $1 \leq j \leq T_c$ **do**
- 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
- 7: Initialize $\mathbf{dK}_j = (0)_{B_c \times d}, \mathbf{dV}_j = (0)_{B_c \times d}$ on SRAM.
- 8: **for** $1 \leq i \leq T_r$ **do**
- 9: Load $\mathbf{Q}_i, \mathbf{O}_i, \mathbf{dO}_i, \mathbf{dQ}_i, L_i, D_i$ from HBM to on-chip SRAM.
- 10: On chip, compute $\mathbf{S}_i^{(j)} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 11: On chip, compute $\mathbf{P}_i^{(j)} = \exp(\mathbf{S}_i^{(j)} - L_i) \in \mathbb{R}^{B_r \times B_c}$.
- 12: On chip, compute $\mathbf{dV}_j \leftarrow \mathbf{dV}_j + (\mathbf{P}_i^{(j)})^\top \mathbf{dO}_i \in \mathbb{R}^{B_c \times d}$.
- 13: On chip, compute $\mathbf{dP}_i^{(j)} = \mathbf{dO}_i \mathbf{V}_j^\top \in \mathbb{R}^{B_r \times B_c}$.
- 14: On chip, compute $\mathbf{dS}_i^{(j)} = \mathbf{P}_i^{(j)} \circ (\mathbf{dP}_i^{(j)} - D_i) \in \mathbb{R}^{B_r \times B_c}$.
- 15: Load $\mathbf{dQ}_i$ from HBM to SRAM, then on chip, update $\mathbf{dQ}_i \leftarrow \mathbf{dQ}_i + \mathbf{dS}_i^{(j)} \mathbf{K}_j \in \mathbb{R}^{B_r \times d}$, and write back to HBM.
- 16: On chip, compute $\mathbf{dK}_j \leftarrow \mathbf{dK}_j + \mathbf{dS}_i^{(j)\top} \mathbf{Q}_i \in \mathbb{R}^{B_c \times d}$.
- 17: **end for**
- 18: Write $\mathbf{dK}_j, \mathbf{dV}_j$ to HBM.
- 19: **end for**
- 20: Return $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$.

Time For Code

- GitHub repo: [Code and explanation for FlashAttention2](#)

Serving LLMs



Guido Appenzeller • 2nd

[+ Follow](#) [...](#)

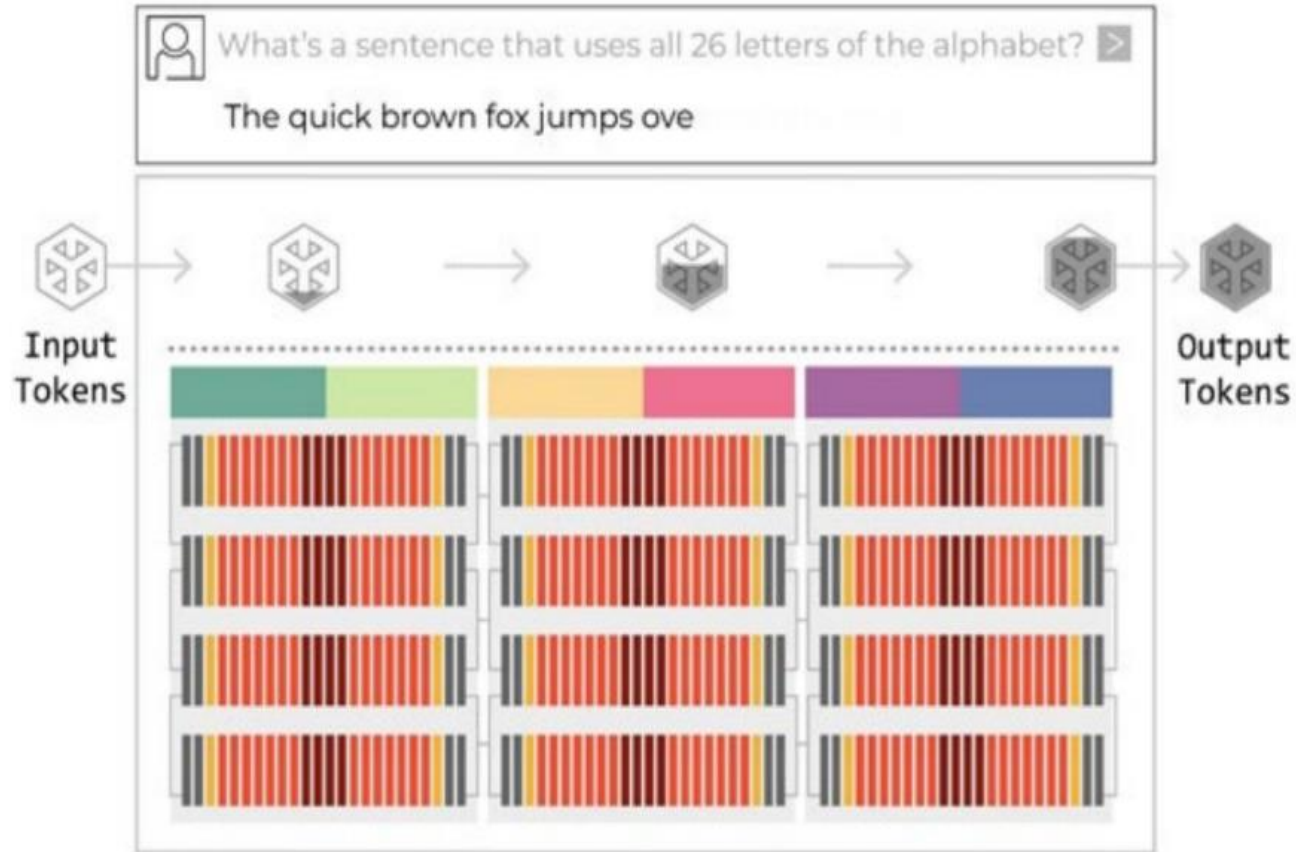
Investing in Infra & AI at a16z. Previously CTO @ Intel & VMwar...

2mo •

Generative AI is becoming crazy cheap. The cost of LLM inference has come down by 100x over 2 years (~\$50 to \$0.50 per 1M tokens) and having an LLM listen to everything you say for the entire year costs about \$3.50.

Where is the next 100x speed up?

Hardware super specialized for Transformer LLMs



Methods for Optimizing Generative AI

- Flash Attention → GPU Optimization
- Paged Attention → Memory Optimization
- vLLM
- Zero(Deepspeed)
- Lightseq
- Ring Attention
- Megatron LM
- Kimi Delta Attention
- LMCache

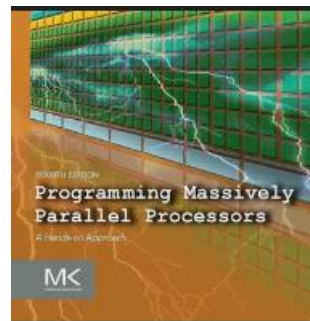
Active Area of Research in LLM:

Efficient Training and Inference

References

- **Textbooks**

[Programming Massively Parallel Processors](#)



- **Papers**

[Attention Is All You Need](#)

[FlashAttention2](#)

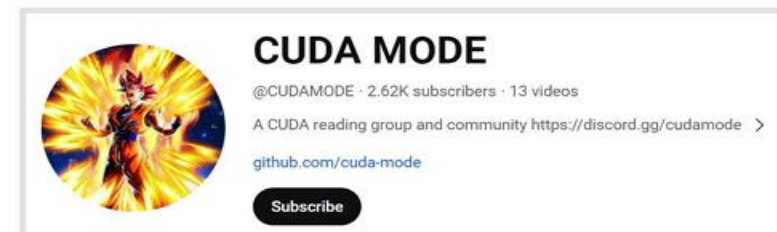
- **Courses**

CMU → Large Language Model Systems

Stanford → Language Model from Scratch

IIT Delhi → Advanced LLMs

Community



CUDA Mode group

Thank You